Informática en la Nube · Parcial 2

NubePaaS

Plataforma como Servicio multi-tenant tipo Render sobre Kubernetes

Dos clientes con stacks distintos · base de datos administrada · HTTPS con OpenSSL · panel web de configuración · despliegue desde Git · monitoreo y comparación de costos

Estudiante	[completar]
Carné	[completar]
Docente	[completar]
Repositorio Git	https://github.com/maxi300/paas-parcial2
Video demo	[enlace al video]
Fecha	25 de octubre de 2026

1. Resumen y cumplimiento de requisitos

Se construyó **NubePaaS**, una plataforma como servicio (PaaS) sobre un clúster **minikube de 2 nodos**. Funciona como Render: el cliente **da solo 4 valores** (nombre/dominio, repositorio Git, runtime/puerto y motor de base de datos) y la plataforma crea todo lo demás con un **chart de Helm**: namespace aislado, cuota del plan, certificado SSL, base de datos con volumen persistente y contraseñas en Secrets, Ingress con HTTPS, autoescalado y middleware.

Se desplegaron **dos clientes con stacks diferentes** (Node.js + PostgreSQL y PHP + MySQL) y, como prueba del modelo de autoservicio, un **tercer cliente creado desde el portal** (TecnoMorazán) sin tocar la terminal.

Requisito del parcial	Cómo se cumple	Sección
2 apps de clientes en 2 tenants, stacks distintos	`portal-clientes`: Node.js 22 + Express + PostgreSQL 16 · `pos-ventas`: PHP 8.3 + Apache + MySQL 8.4	4
Base de datos, usuarios, login, variables de entorno	Registro/login (bcrypt / password_hash, JWT y sesión httpOnly); credenciales inyectadas desde Secret	4, 6
Dockerfile e imágenes listas	Un Dockerfile por app; imágenes construidas en ambos nodos y desde Git	4, 10
Panel web para editar values.yaml y aplicar	NubePaaS: panel del dueño (/admin) y portal del cliente; aplica con `helm upgrade`	9
Links públicos HTTP y HTTPS · certificado OpenSSL como recurso K8s	CA propia → certificados por cliente → Secrets `kubernetes.io/tls`; acceso desde Windows por VirtualBox (puente)	7, 8
Base de datos administrada con Ingress	StatefulSet + PVC + Secret; Adminer publicado en `db.<cliente>.paas.local` con HTTPS	6
Wappalyzer activo en cada app	Capturas en Alpha, Beta, TecnoMorazán, Adminer y NubePaaS	11
CRUD en PHP y API REST en Node	Panel PHP de platillos · panel de productos que consume `GET/POST/PUT/DELETE /api/products`	12
Aislamiento entre clientes	NetworkPolicy por namespace: prueba PERMITIDO / BLOQUEADO; el portal impide ver otro cliente	12, 13
Despliegue desde Git (tipo Render)	Botón «Desplegar último commit»: git clone → build → helm upgrade; probado con los commits 9bfbe29 y 1746196	10
Escalado automático	HPA por plan; demostración en vivo: Alpha de 2 a 5 réplicas bajo carga sin afectar a Beta	13
Comparación de costos Cliente 1 vs 2 (Lab 2)	Grafana + prueba de carga: solicitudes/min, CPU, RAM, disco, velocidad, precio del plan y costo real	14
Repositorio Git centralizado	github.com/maxi300/paas-parcial2 (apps, chart, panel, pruebas, monitoreo; sin llaves)	16

2. Arquitectura

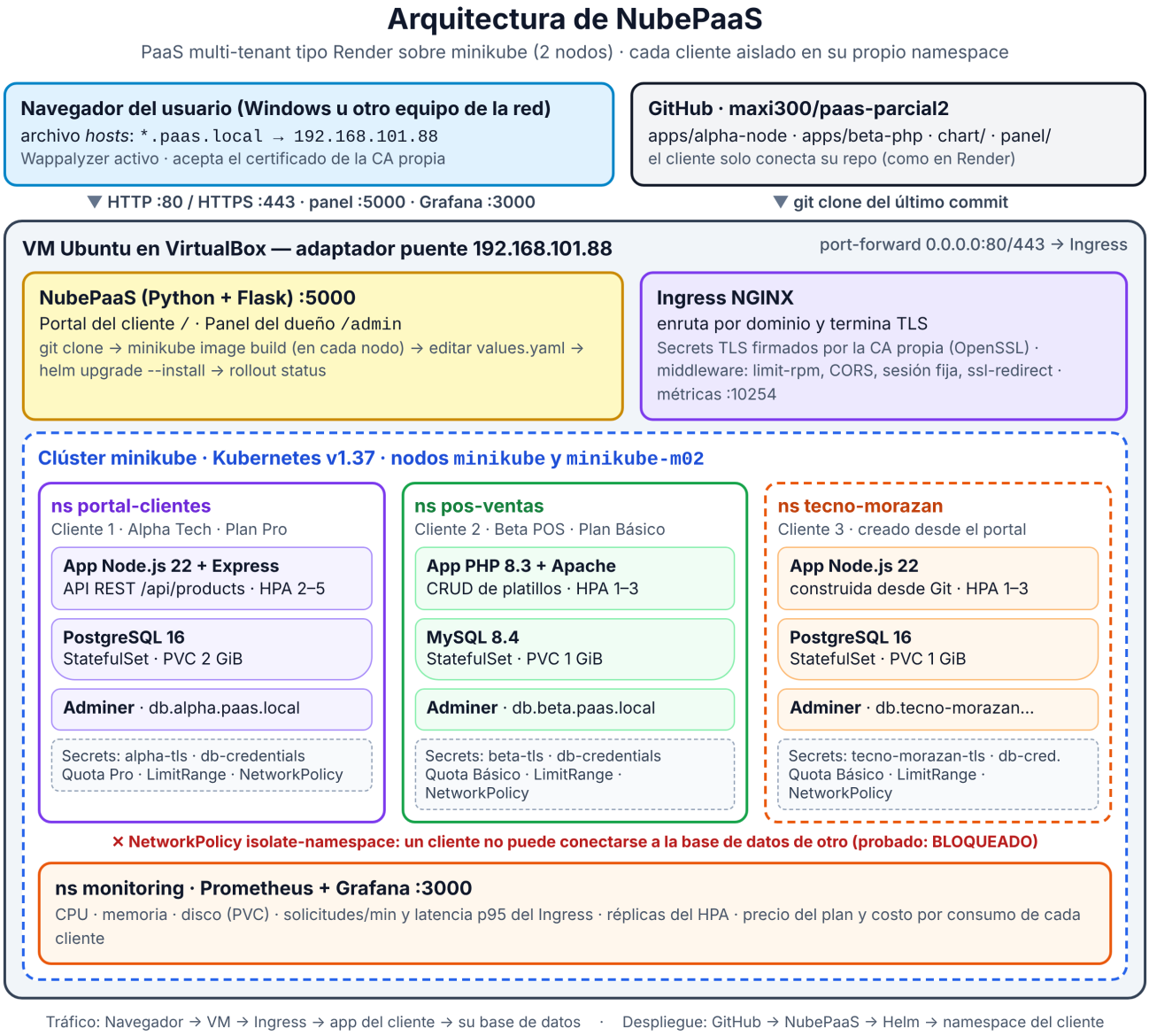


Figura 1. Arquitectura de NubePaaS: cada cliente vive en su namespace con su app, su base de datos, su Adminer, sus Secrets y su cuota; NubePaaS despliega desde GitHub con Helm.

2.1 Componentes

Componente	Tecnología	Función
Clúster	minikube v1.39 · Kubernetes v1.37 · containerd 2.3 · 2 nodos (`minikube`, `minikube-m02`)	Ejecuta todas las cargas
Plantilla de despliegue	Helm · chart `paas-app-chart` v0.2.0	Un release por cliente (`mi-app`, `mi-app-pos`, `tecno-morazan`)
Entrada	Ingress NGINX (addon de minikube)	Enruta por dominio, termina TLS, aplica middleware y expone métricas
Seguridad	OpenSSL (CA propia) · Secrets TLS · NetworkPolicy · ResourceQuota · LimitRange	HTTPS, aislamiento y límites por plan
Plataforma	NubePaaS (Python 3 + Flask)	Portal del cliente tipo Render y panel del dueño
Observabilidad	kube-prometheus-stack (Prometheus + Grafana + kube-state-metrics)	Consumo, solicitudes, latencia y costos por cliente
Código	GitHub `maxi300/paas-parcial2`	Fuente de los despliegues y entregable

2.2 El modelo de los «4 valores»

Lo que da el cliente	Lo que agrega la plataforma automáticamente
1. Nombre / dominio	Namespace aislado, cuota del plan, LimitRange y NetworkPolicy
2. Repositorio Git (+ rama y carpeta del Dockerfile)	Construcción de la imagen en los nodos y despliegue con Helm
3. Runtime y puerto	Chequeos de salud (liveness/readiness) según el runtime, HPA y recursos del plan
4. Motor de base de datos	StatefulSet con PVC, contraseñas aleatorias en Secret, Adminer con HTTPS y certificado SSL propio

3. Entorno y preparación del clúster

El clúster corre dentro de una VM Ubuntu en VirtualBox con **adaptador puente** (IP **192.168.101.88**). Los dos nodos están `Ready` y los releases de Helm de los laboratorios anteriores se reutilizaron (`mi-app` y `mi-app-pos`) pasando al chart **v2**. Antes del despliegue se validó el chart sin tocar el clúster:

```
$ helm lint . -f values-portal.yaml
1 chart(s) linted, 0 chart(s) failed
$ helm lint . -f values-pos.yaml
1 chart(s) linted, 0 chart(s) failed
$ helm template mi-app . -f values-portal.yaml -n portal-clientes | grep -E "^kind:|image:|
host:|secretName:|limit-rpm"
kind: Secret / Service ×3 / Deployment ×2 (app + adminer) / HorizontalPodAutoscaler /
StatefulSet / Ingress ×2
image: "alpha-node-app:1.0" · image: postgres:16-alpine · image: adminer:4.8.1
host: "alpha.paas.local" · host: "db.alpha.paas.local" · secretName: alpha-tls · limit-rpm:
"600"
$ helm template mi-app-pos . -f values-pos.yaml -n pos-ventas | ...
image: "beta-php-app:1.0" · image: mysql:8.4 · host: "beta.paas.local" · secretName: beta-tls ·
limit-rpm: "120"
```

Ajustes previos: HSTS desactivado en el ConfigMap de ingress-nginx (para poder mostrar HTTP y HTTPS), las apps viejas del laboratorio escaladas a 0 (sin borrarlas) para liberar cuota, y Prometheus aligerado (retención de 12 h, recolección cada 60 s, Alertmanager apagado). Ver la sección 15.

4. Aplicaciones de los clientes

	Cliente 1 · Alpha Tech	Cliente 2 · Beta POS
Namespace / release	`portal-clientes` / `mi-app`	`pos-ventas` / `mi-app-pos`
Giro	Tienda de tecnología	Punto de venta de restaurante
Runtime	Node.js 22 + Express 4 (puerto 3000)	PHP 8.3 + Apache 2.4 (puerto 8080)
Base de datos	PostgreSQL 16 · PVC 2 GiB	MySQL 8.4 · PVC 1 GiB
Autenticación	bcrypt + JWT en cookie httpOnly	password_hash() + sesión PHP httpOnly + CSRF
Funcionalidad	Catálogo + **API REST** `/api/products` (CRUD con JWT)	Menú del día + **CRUD** de platillos (panel.php)
Interfaz	Bootstrap 5	Bulma 1.0
Plan	Pro · HPA 2–5 · 600 sol/min · CORS	Básico · HPA 1–3 · 120 sol/min · sesión fija

4.1 Dockerfiles

	alpha-node (apps/alpha-node/Dockerfile)	beta-php (apps/beta-php/Dockerfile)
Imagen base	`node:22-alpine`	`php:8.3-apache`
Pasos	copia package.json → instala dependencias de producción → copia `src/` → usuario sin privilegios `node`	instala `pdo_mysql` → php.ini de producción (expose_php=On, cookies httpOnly) → Apache en 8080 → copia `src/`
Arranque	`node src/server.js` (EXPOSE 3000)	Apache (EXPOSE 8080)

Toda la configuración entra por **variables de entorno** (APP_NAME, TENANT, APP_ENV, BRAND_COLOR, DB_HOST, DB_PORT, DB_NAME); **DB_USER, DB_PASSWORD y JWT_SECRET vienen de un Secret**, nunca del código ni del repositorio.

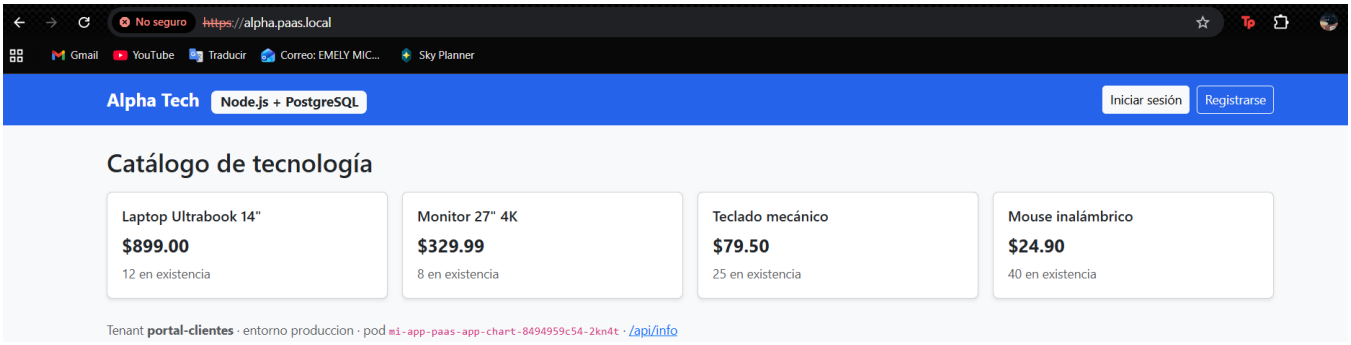


Figura 2. Cliente 1 (Alpha Tech, Node.js + PostgreSQL): catálogo servido por HTTPS desde Windows.

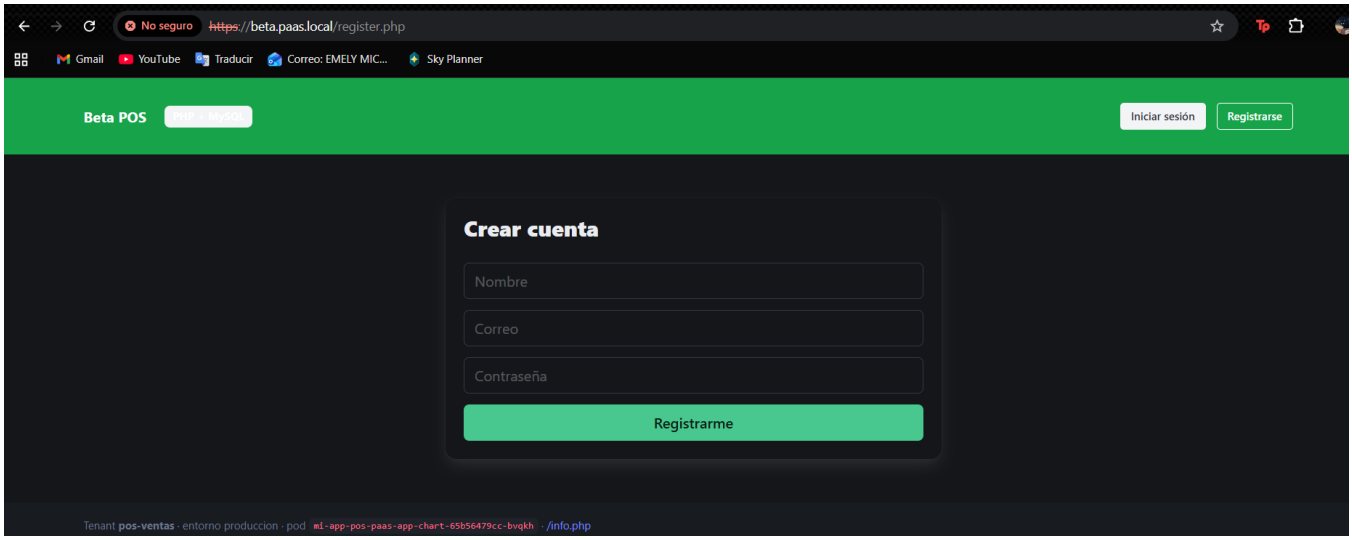


Figura 3. Cliente 2 (Beta POS, PHP + MySQL): registro servido por HTTPS desde Windows.

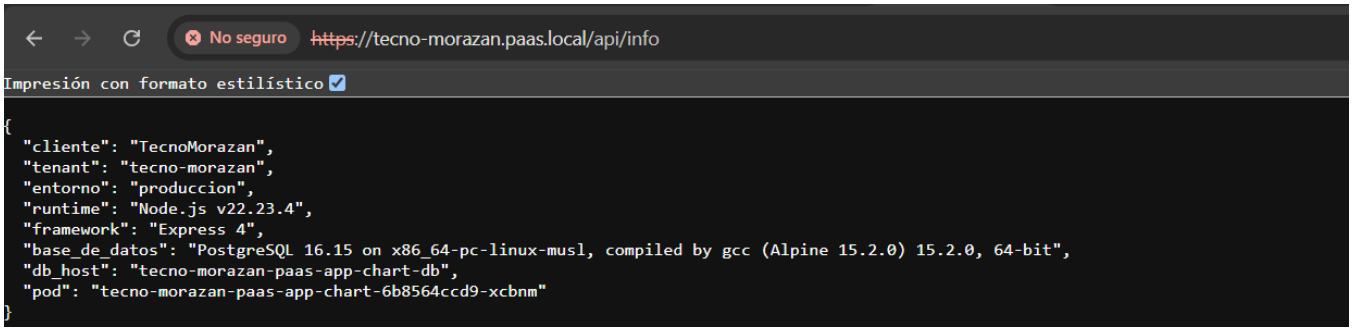


Figura 4. Endpoint /api/info de TecnoMorazán (Node.js 22.23.4 + PostgreSQL 16.15) indicando el pod que respondió.

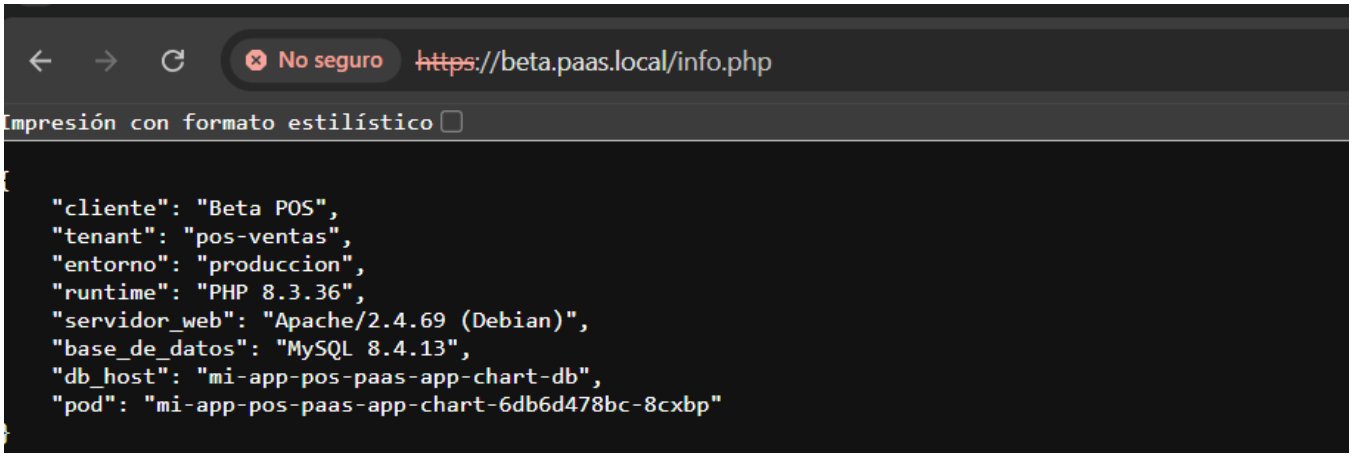


Figura 5. Endpoint /info.php de Beta POS: PHP 8.3.36, Apache 2.4.69 y MySQL 8.4.13.

5. Chart de Helm v2 (paas-app-chart 0.2.0)

Se partió del chart de los laboratorios y se agregaron plantillas para la base de datos, sus credenciales y Adminer. El Deployment lleva la anotación `paas/config-hash` (hash de los values): cualquier cambio en el panel recrea los pods de forma ordenada.

Plantilla	Qué crea
deployment.yaml	La app con su imagen, puerto, variables de entorno, Secret de BD, probes y recursos
service.yaml · hpa.yaml	Service ClusterIP y HorizontalPodAutoscaler (CPU objetivo 70 %)
ingress.yaml	Dominio del cliente con TLS y anotaciones de middleware (limit-rpm, CORS, sticky, ssl-redirect, body-size)
database.yaml	Service headless + StatefulSet PostgreSQL o MySQL con PVC y readinessProbe
secret-db.yaml	Usuario, contraseña, contraseña root y JWT aleatorios; se conservan entre upgrades con `lookup`
dbadmin.yaml	Adminer 4.8.1 + Service + Ingress con TLS en `db.<dominio>`

Fragmento de `values-portal.yaml` (el archivo que edita el panel):

```
runtime: node
image: {repository: alpha-node-app, tag: git-1746196}   # la etiqueta la pone «Desplegar desde
Git»
containerPort: 3000
domain: alpha.paas.local
env: {APP_NAME: Alpha Tech, TENANT: portal-clientes, APP_ENV: produccion, BRAND_COLOR:
'#7c3aed'}
autoscaling: {enabled: true, minReplicas: 2, maxReplicas: 5, targetCPU: 70}
resources: {requests: {cpu: 100m, memory: 128Mi}, limits: {cpu: 500m, memory: 256Mi}}
database: {enabled: true, engine: postgres, version: 16-alpine, name: alpha_db, user:
alpha_user, storage: 2Gi}
dbAdmin: {enabled: true}
tls: {enabled: true, secretName: alpha-tls}
middleware: {sslRedirect: false, rateLimitRPM: 600, stickySessions: false, proxyBodySize: 10m,
cors: true}
source: {repo: https://github.com/maxi300/paas-parcial2.git, branch: main, path: apps/alpha-
node}
```

5.1 Despliegue y verificación

```
$ kubectl get deploy,sts,ingress,hpa,pvc -n portal-clientes
deployment.apps/mi-app-paas-app-chart           2/2     ·  deployment.apps/mi-app-paas-app-
chart-dbadmin   1/1
statefulset.apps/mi-app-paas-app-chart-db       1/1
ingress  mi-app-paas-app-chart          alpha.paas.local      192.168.49.2   80, 443
ingress  mi-app-paas-app-chart-dbadmin  db.alpha.paas.local   192.168.49.2   80, 443
hpa      mi-app-paas-app-chart          cpu: 4%/70%   MINPODS 2   MAXPODS 5   REPLICAS 2
pvc      data-mi-app-paas-app-chart-db-0   Bound   2Gi   RWO   standard
$ kubectl get pods -n portal-clientes -o custom-
columns=POD:.metadata.name,IMAGEN:.spec.containers[0].image
mi-app-paas-app-chart-…-2kn4t   alpha-node-app:1.0   ·   mi-app-paas-app-chart-db-0
postgres:16-alpine
mi-app-paas-app-chart-…-8fmlq   alpha-node-app:1.0   ·   mi-app-paas-app-chart-dbadmin-…
adminer:4.8.1
$ kubectl get deploy,sts,ingress,hpa,pvc -n pos-ventas
deployment mi-app-pos-paas-app-chart 1/1 · dbadmin 1/1 · statefulset mi-app-pos-paas-app-chart-
db 1/1
ingress beta.paas.local / db.beta.paas.local (80, 443) · hpa cpu: 2%/70% (1–3) · pvc 1Gi Bound
```

Las dos peticiones HTTP y HTTPS de la verificación las atendieron pods distintos (`…-2kn4t` y `…-8fmlq`): el Service **balancea la carga** entre las 2 réplicas del Cliente 1.

6. Base de datos administrada

Cada cliente recibe su base en un **StatefulSet con volumen persistente**. Las contraseñas se generan al instalar, se guardan en el Secret `<release>-paas-app-chart-db-credentials` y se inyectan en la app. Para administrarla, la plataforma publica **Adminer** con su propio dominio y HTTPS. El cliente ve sus datos de conexión en el portal (la contraseña queda oculta hasta presionar «Mostrar»).

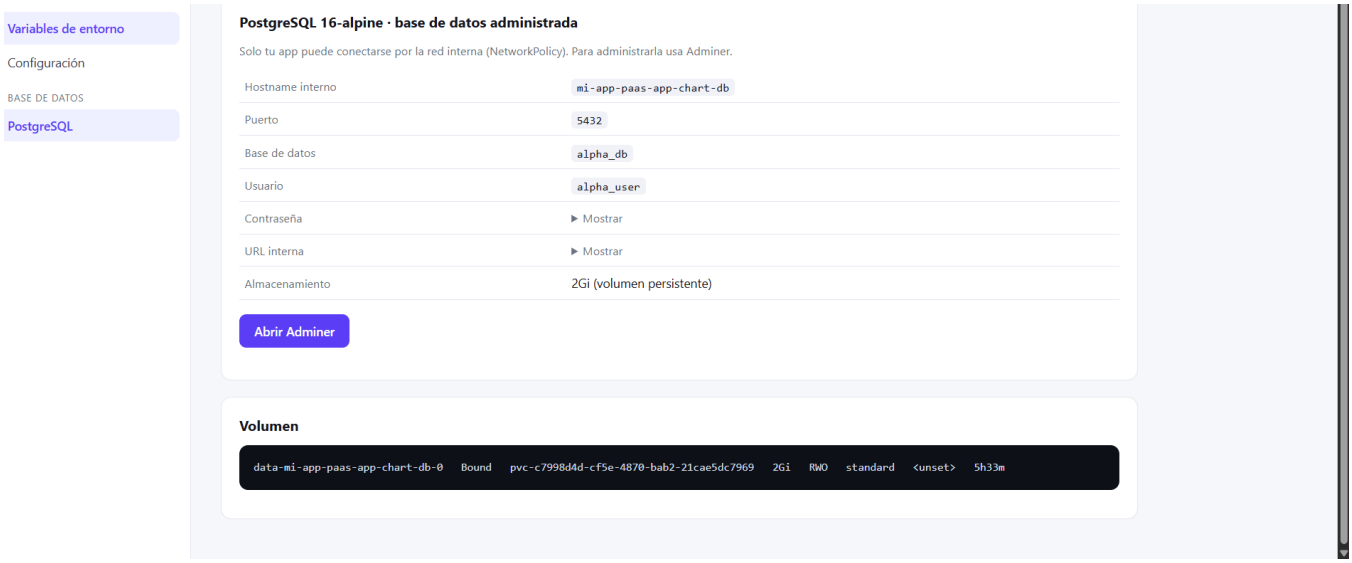


Figura 6. Portal del cliente Alpha: base PostgreSQL 16 con hostname interno, puerto, usuario, contraseña oculta y volumen de 2 GiB.

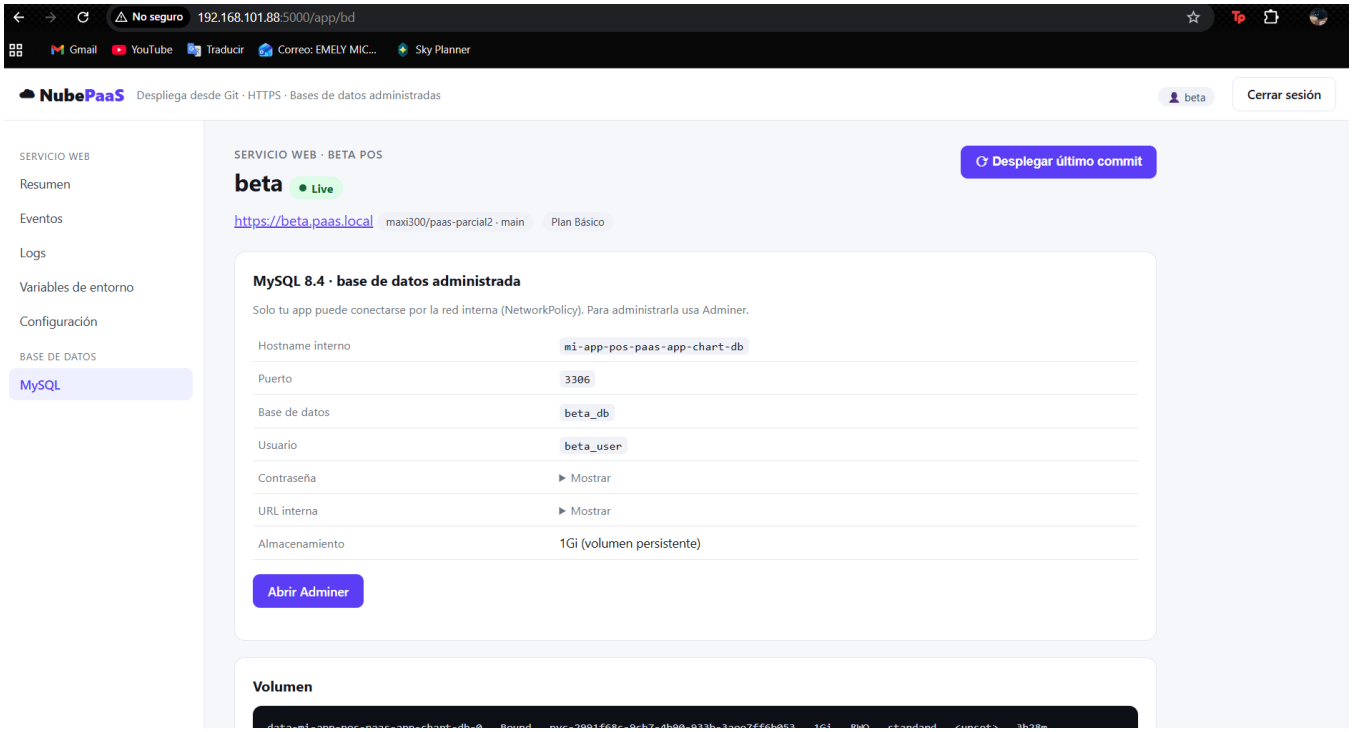


Figura 7. Portal del cliente Beta: base MySQL 8.4 con su volumen de 1 GiB.



Figura 8. Adminer publicado por Ingress con HTTPS en db.alpha.paas.local (cada cliente tiene el suyo).

La primera inicialización de MySQL tardó unos 2 minutos (descarga de 238 MB e inicialización de InnoDB); durante ese tiempo la app PHP permaneció «no lista» gracias a su readinessProbe `/health.php?ready=1`, y el pod anterior siguió atendiendo (actualización sin caída).

6.1 Datos de conexión de cada base (Adminer)

Para entrar a Adminer se abre `https://db.<cliente>.paas.local` y se llenan estos datos. **El motor debe coincidir** con el del cliente (Beta usa MySQL, no PostgreSQL). El servidor es el nombre interno del Service de la base, que solo existe dentro del namespace del cliente.

Cliente · Adminer	Motor	Servidor	Usuario	Base de datos
Alpha Tech · db.alpha.paas.local	PostgreSQL (5432)	`mi-app-paas-app-chart-db`	`alpha_user`	`alpha_db`
Beta POS · db.beta.paas.local	**MySQL** (3306)	`mi-app-pos-paas-app-chart-db`	`beta_user`	`beta_db`
TecnoMorazán · db.tecno-morazan.paas.local	PostgreSQL (5432)	`tecno-morazan-paas-app-chart-db`	`tecno_morazan_user`	`tecno_morazan_db`

Las contraseñas no se escriben en el informe ni en Git: Helm las genera al azar (24 caracteres) en el Secret `<release>-paas-app-chart-db-credentials`. El cliente la ve en su portal (**Base de datos** → **Contraseña** → **Mostrar**) y el dueño puede leerla con:

```
kubectl get secret mi-app-paas-app-chart-db-credentials -n portal-clientes -o
jsonpath='{.data.DB_PASSWORD}' | base64 -d; echo
kubectl get secret mi-app-pos-paas-app-chart-db-credentials -n pos-ventas -o
jsonpath='{.data.DB_PASSWORD}' | base64 -d; echo
kubectl get secret tecno-morazan-paas-app-chart-db-credentials -n tecno-morazan -o
jsonpath='{.data.DB_PASSWORD}' | base64 -d; echo
```

7. HTTPS con certificado propio (OpenSSL)

Se creó una **autoridad certificadora propia** y, con ella, un certificado por cliente que cubre su app y su Adminer. Las llaves quedan en `~/certs`, fuera del repositorio (bloqueadas por `.gitignore`).

```
openssl genrsa -out ca.key 4096
openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -subj "/C=SV/O=PaaS Local/CN=PaaS
Local Root CA" -out ca.crt
for c in alpha beta; do
  openssl genrsa -out $c.key 2048
  openssl req -new -key $c.key -subj "/C=SV/O=PaaS Local/CN=$c.paas.local" -out $c.csr
  printf "subjectAltName=DNS:$c.paas.local,DNS:db.$c.paas.local\n..." > $c.ext
```

```
  openssl x509 -req -in $c.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out $c.crt -days 365 -
sha256 -extfile $c.ext
done
$ openssl x509 -in alpha.crt -noout -subject -issuer -dates -ext subjectAltName
subject=C=SV, O=PaaS Local, CN=alpha.paas.local
issuer=C=SV, O=PaaS Local, CN=PaaS Local Root CA
notBefore=Oct 24 21:43:52 2026 GMT   notAfter=Oct 24 21:43:52 2027 GMT
X509v3 Subject Alternative Name: DNS:alpha.paas.local, DNS:db.alpha.paas.local
$ kubectl create secret tls alpha-tls --cert=alpha.crt --key=alpha.key -n portal-clientes   →
secret/alpha-tls created
$ kubectl create secret tls beta-tls  --cert=beta.crt  --key=beta.key  -n pos-ventas       →
secret/beta-tls created
$ kubectl get secret alpha-tls -n portal-clientes   →   alpha-tls   kubernetes.io/tls   2
$ curl -skv https://alpha.paas.local/health  →  subject: CN=alpha.paas.local · issuer: CN=PaaS
Local Root CA · HTTP/2 200
```

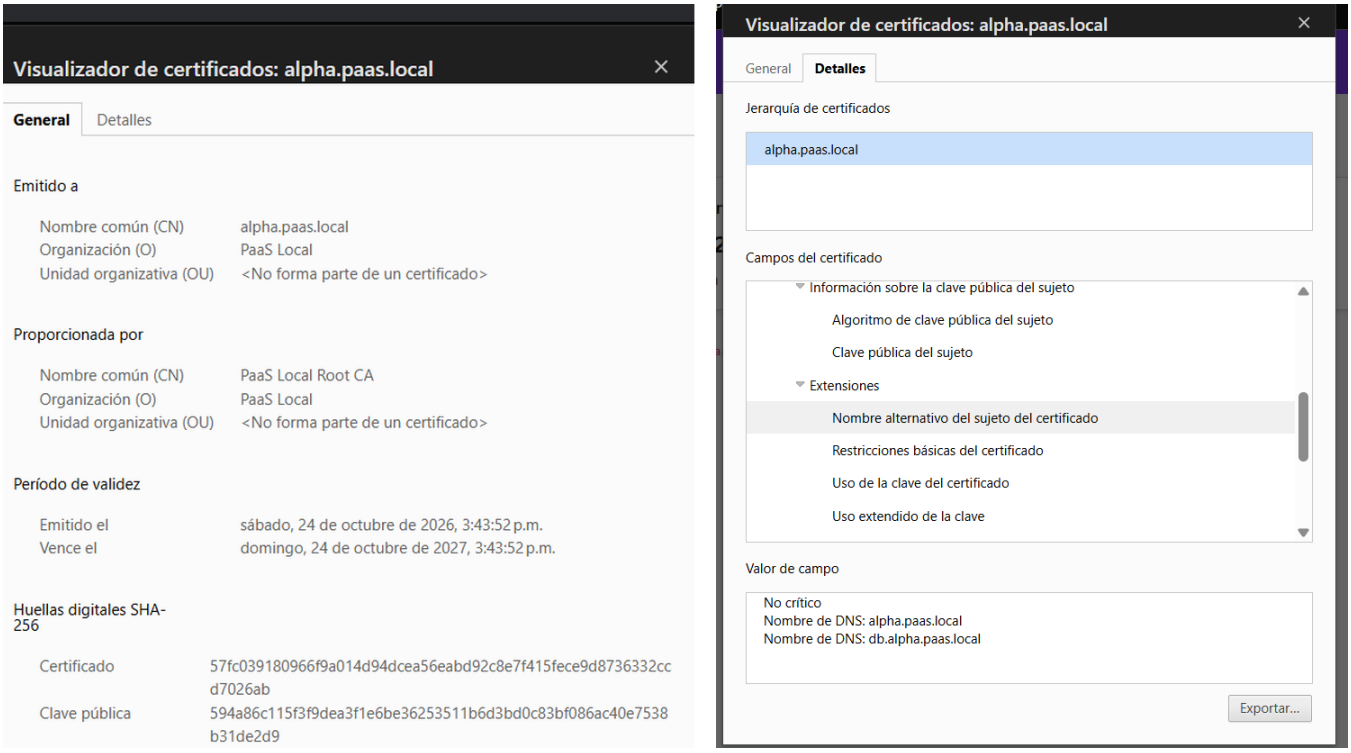


Figura 9. Certificado de alpha.paas.local: emitido por PaaS Local Root CA, válido del 24/10/2026 al 24/10/2027, con los nombres DNS alpha.paas.local y db.alpha.paas.local.

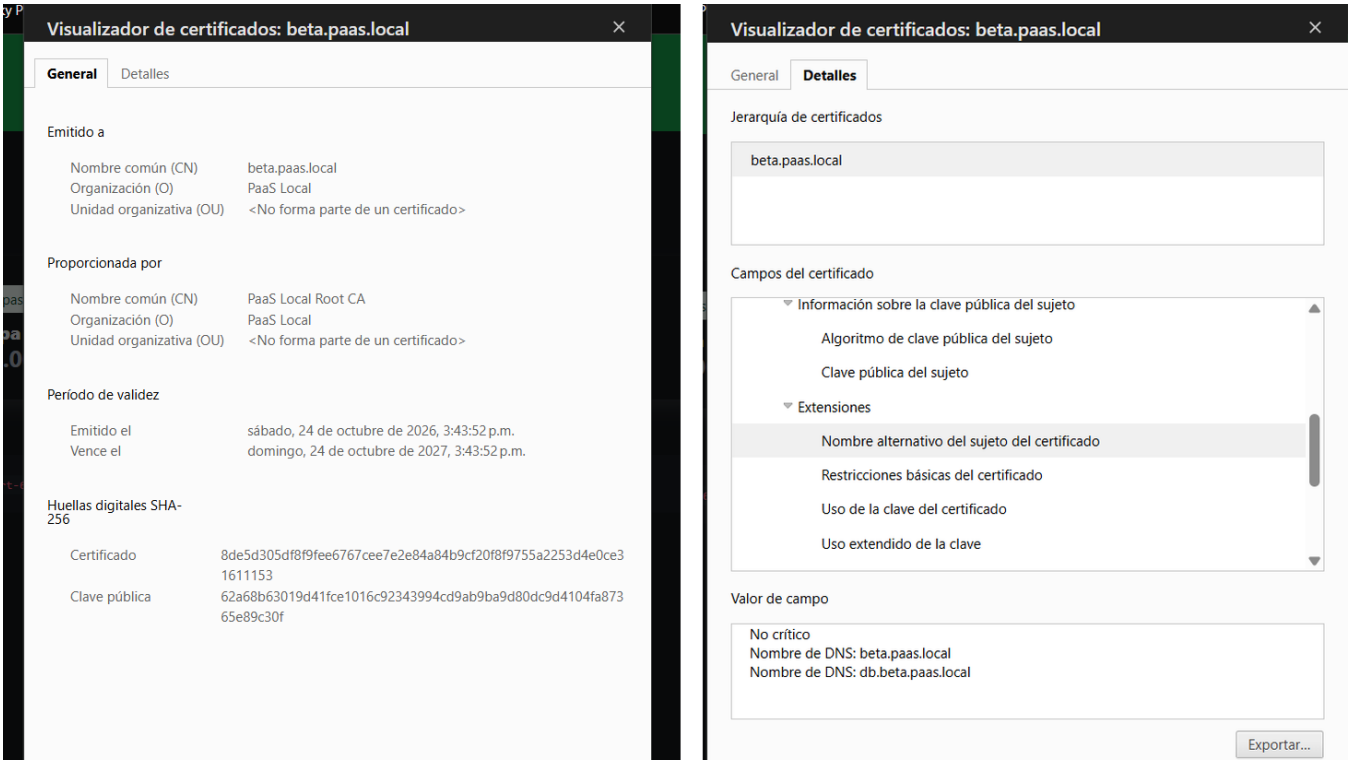


Figura 10. Certificado de beta.paas.local (General y Detalles con sus dos nombres DNS).

El navegador muestra «No seguro» (`NET::ERR_CERT_AUTHORITY_INVALID`) porque la CA no es pública; el cifrado TLS está activo, tal como se indicó en clase. El certificado de TecnoMorazán lo generó la plataforma automáticamente al crear el servicio:

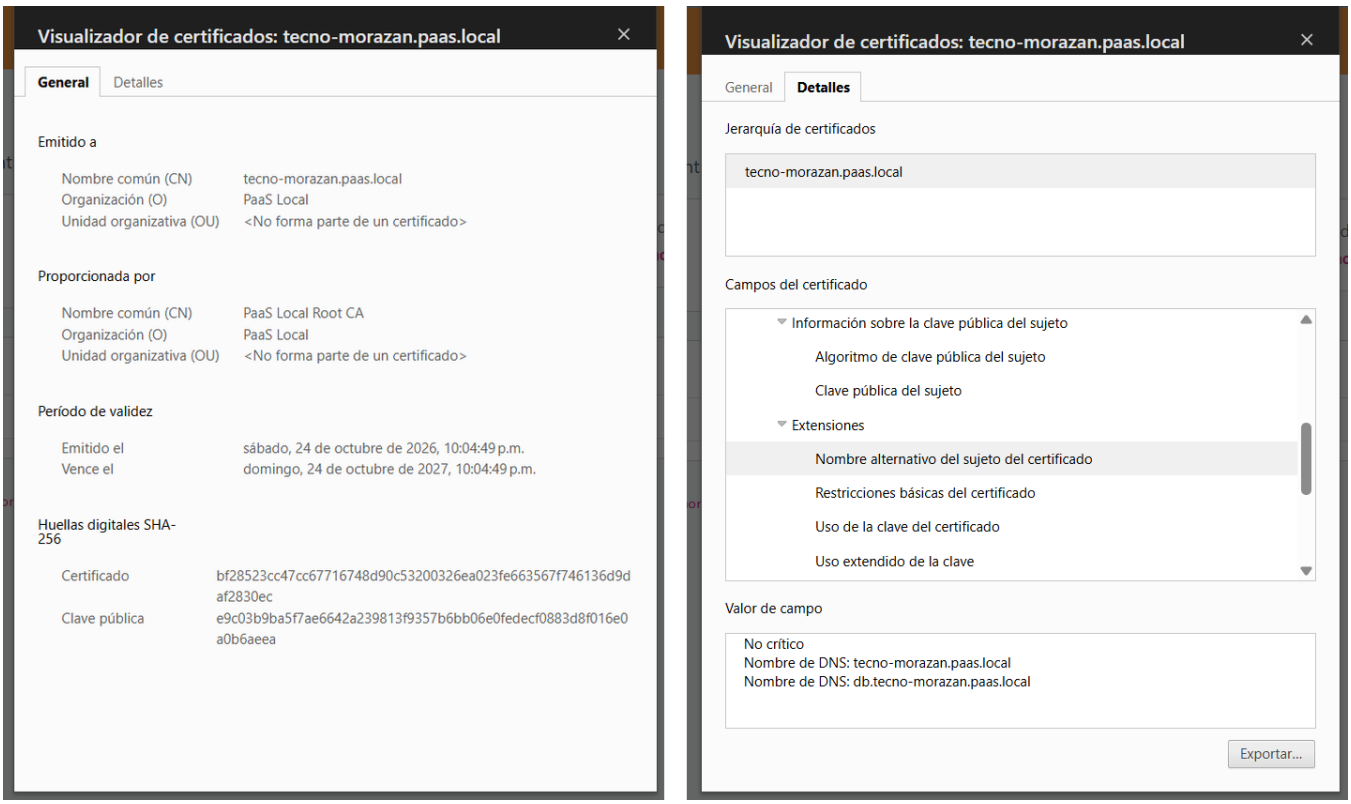


Figura 11. Certificado generado automáticamente por NubePaaS para el cliente nuevo tecno-morazan.paas.local.

8. Acceso público HTTP / HTTPS vía VirtualBox

La VM usa **adaptador puente**, así que tiene una IP de la red local (192.168.101.88). Un port-forward publica el Ingress en los puertos 80 y 443 de la VM, y cualquier equipo de la red llega a los clientes agregando una línea a su archivo hosts.

```
# En la VM (terminal dedicada)
sudo kubectl --kubeconfig /home/max/.kube/config port-forward --address 0.0.0.0 \
     -n ingress-nginx svc/ingress-nginx-controller 80:80 443:443
Forwarding from 0.0.0.0:80 -> 80
Forwarding from 0.0.0.0:443 -> 443

# En Windows (PowerShell como administrador)
Add-Content C:\Windows\System32\drivers\etc\hosts "192.168.101.88 alpha.paas.local
db.alpha.paas.local beta.paas.local db.beta.paas.local"
ping -n 1 alpha.paas.local   →   Respuesta desde 192.168.101.88: tiempo=5ms TTL=64
```



Figura 12. PowerShell en Windows: se agregan los dominios al archivo hosts y el ping responde desde la IP de la VM.

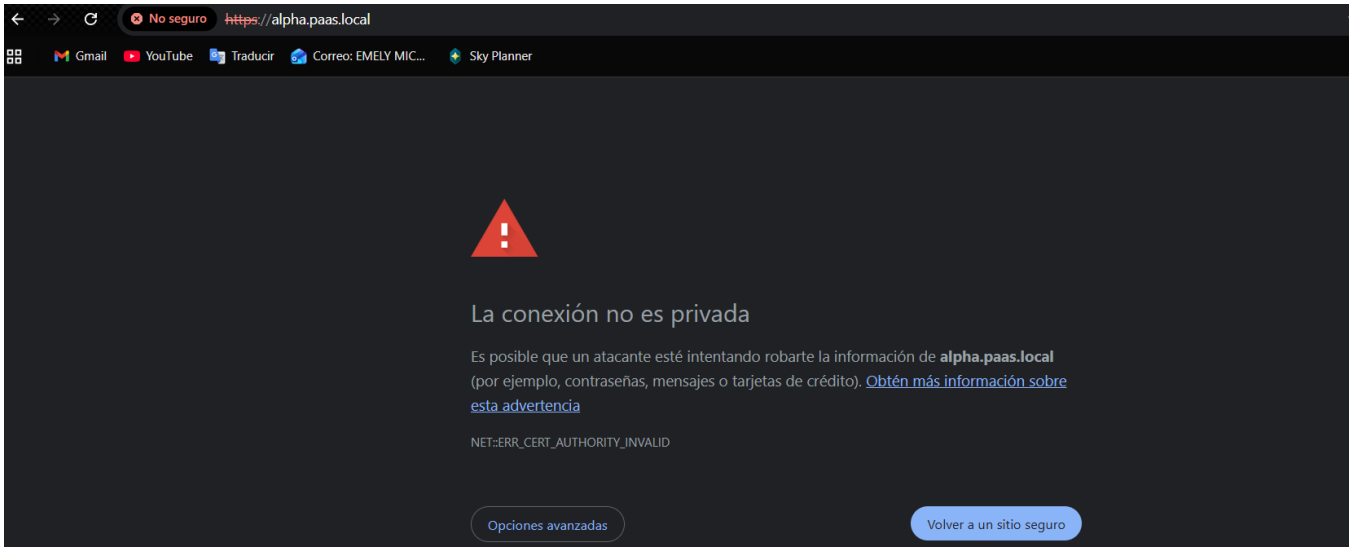


Figura 13. Advertencia del navegador por la CA propia (NET::ERR_CERT_AUTHORITY_INVALID): el cifrado TLS está activo.

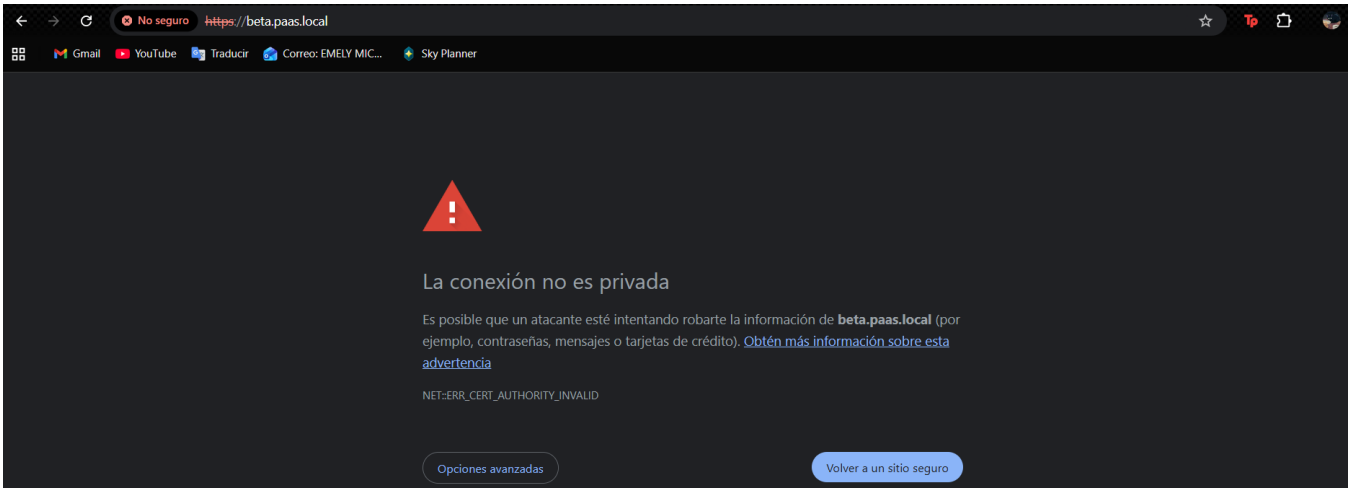


Figura 14. Cliente 2 (Beta POS): el menú del día por HTTPS desde Windows.

9. Panel web NubePaaS

La plataforma tiene **dos portales separados**, con diseños distintos, servidos en `http://192.168.101.88:5000`:

	Portal del cliente (/)	Panel del dueño (/admin)
Acceso	Inicio de sesión propio por cliente; registro público	Solo el usuario admin (autenticación HTTP)
Ve	Solo su servicio	Todos los clientes
Puede	Desplegar desde Git, ver eventos y logs, editar variables, repo/rama/runtime/puerto, ver credenciales de su BD	Todo lo anterior + HPA, recursos, SSL, middleware, dominio, editor YAML completo, eliminar clientes

9.1 Panel del dueño: editar values.yaml y aplicar

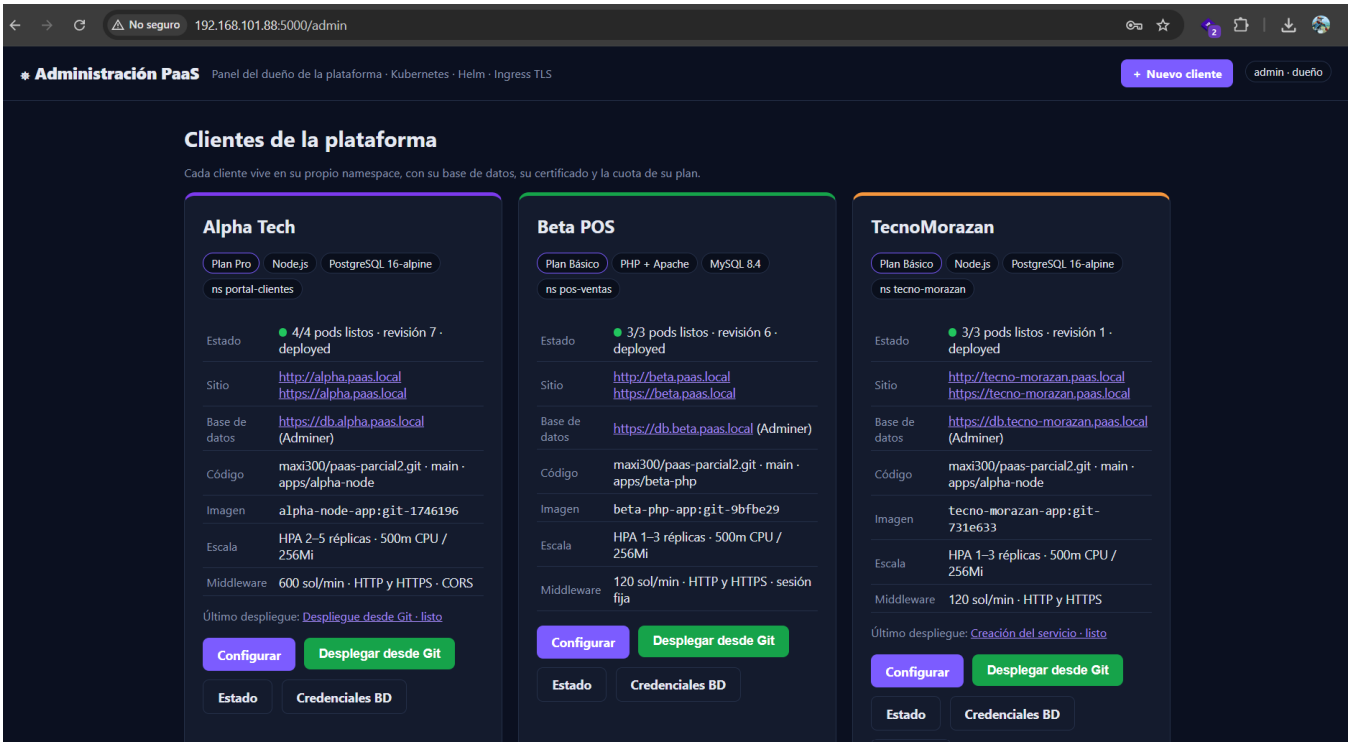


Figura 15. Panel del dueño con los 3 clientes: plan, stack, estado de pods, revisión de Helm, links, repositorio, imagen, escala y middleware.

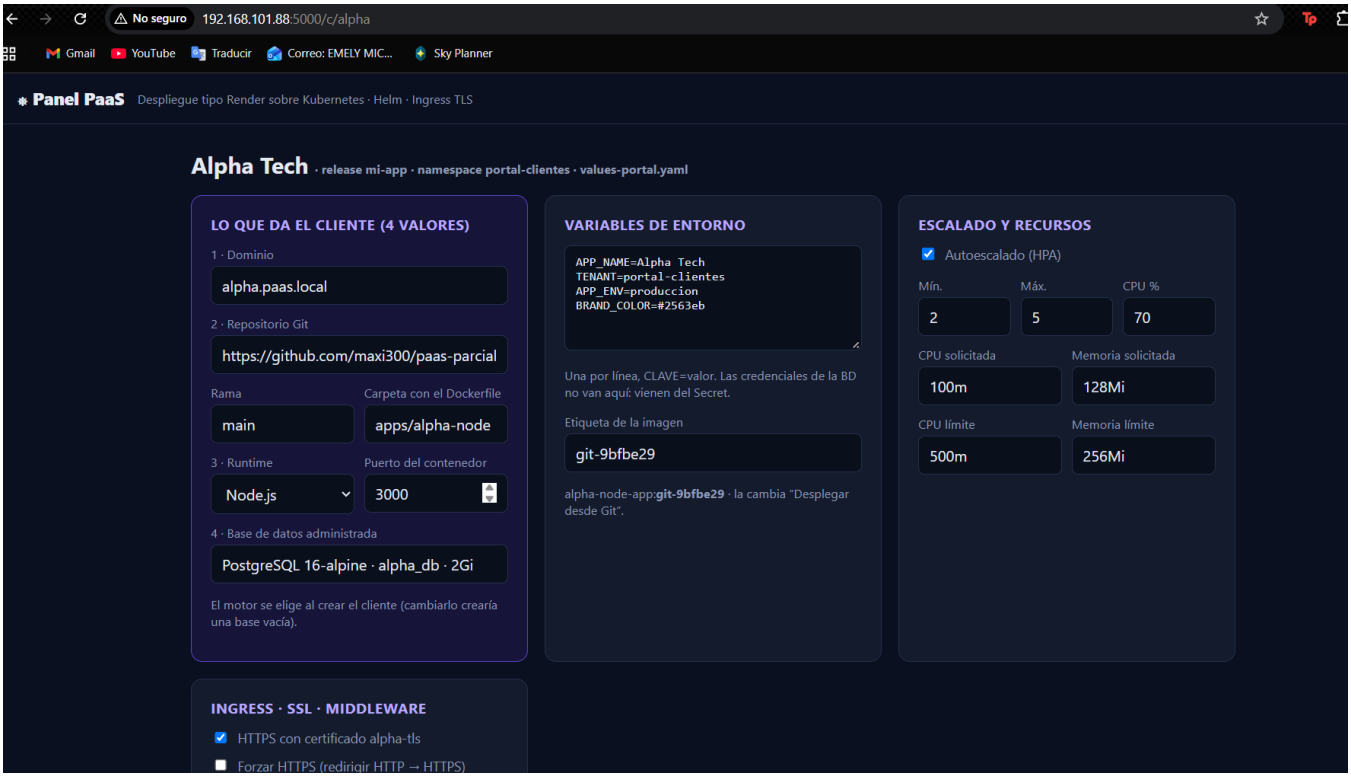


Figura 16. Formulario de configuración: los 4 valores del cliente, variables de entorno, escalado y recursos.



Figura 17. Ingress / SSL / middleware y el values.yaml completo, editable y aplicable desde el panel.

Demostración en vivo: se cambió `BRAND_COLOR` de `#2563eb` a `#7c3aed` y se presionó «Guardar y aplicar». El panel ejecutó `helm upgrade --wait` y `kubectl rollout status`; los pods se recrearon y la app cambió de color:

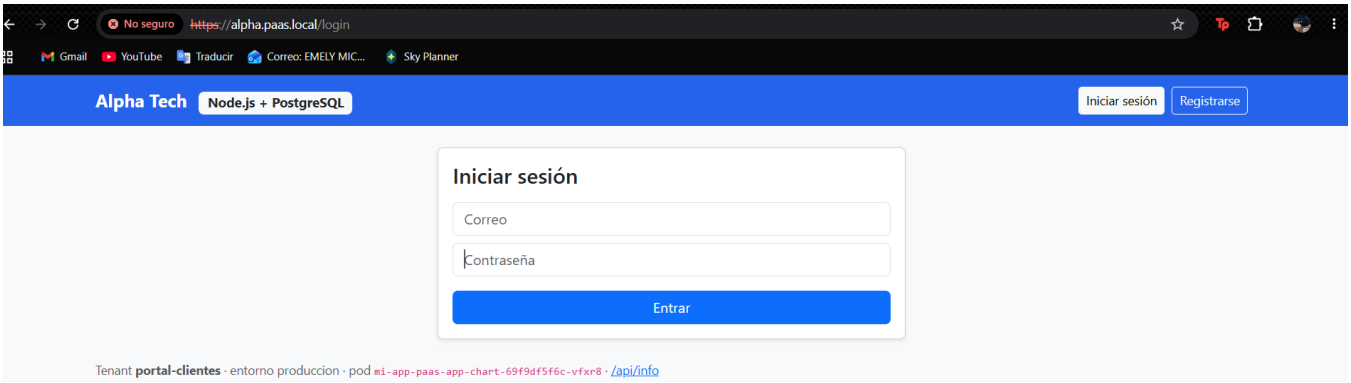


Figura 18. Antes: Alpha Tech con BRAND_COLOR #2563eb (azul).

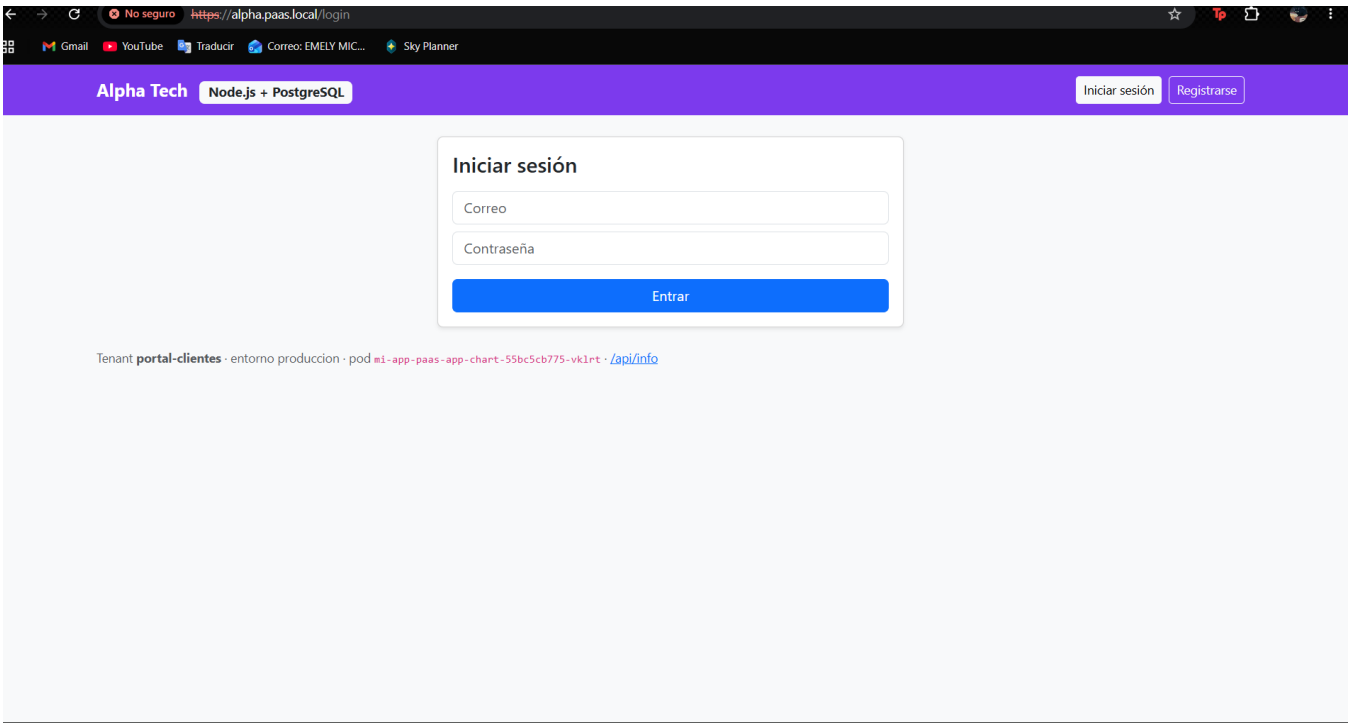


Figura 19. Después de «Guardar y aplicar»: color #7c3aed (morado); el pie de página muestra un pod nuevo.

9.2 Portal del cliente (experiencia tipo Render)

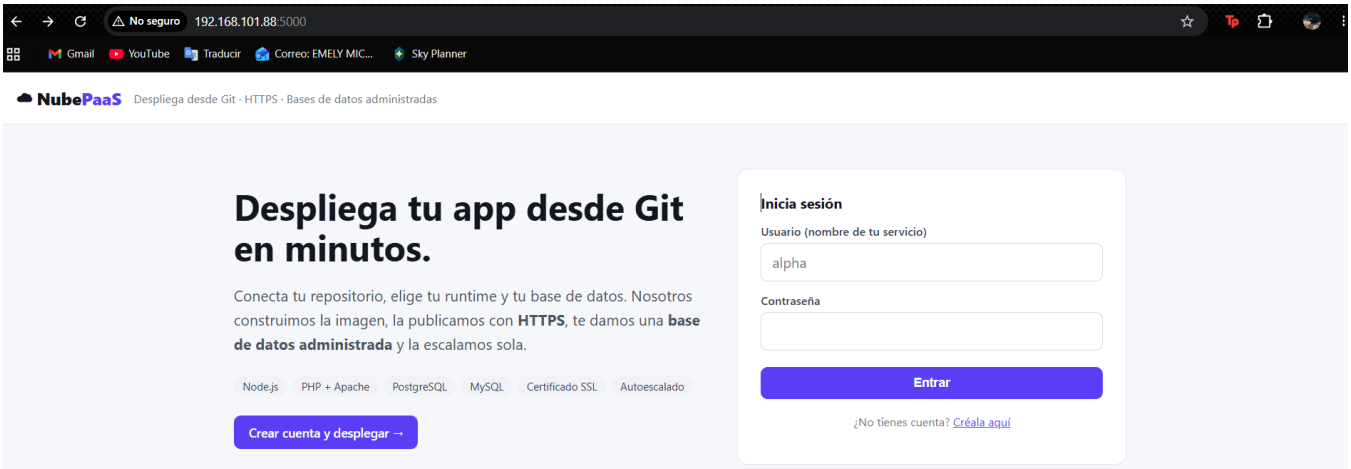


Figura 20. Portada de NubePaaS: «Despliega tu app desde Git en minutos» con inicio de sesión del cliente.

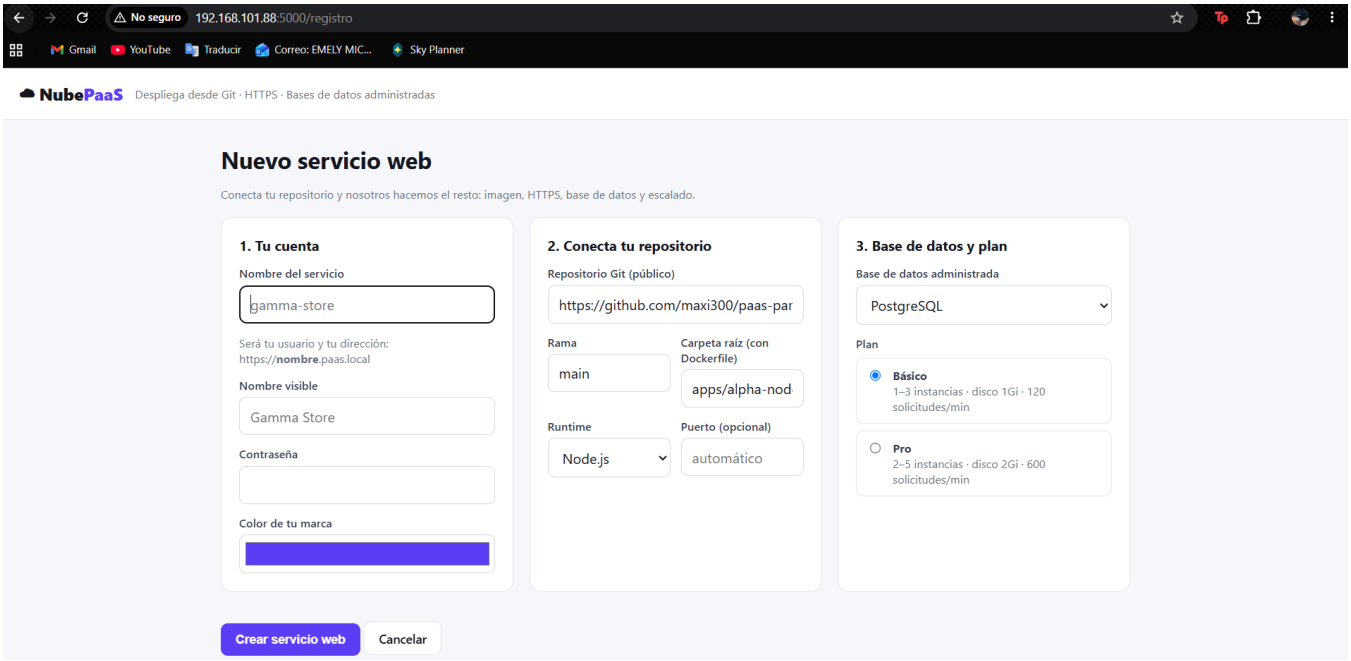


Figura 21. Formulario «Nuevo servicio web»: cuenta, repositorio Git, base de datos y plan (los 4 valores).

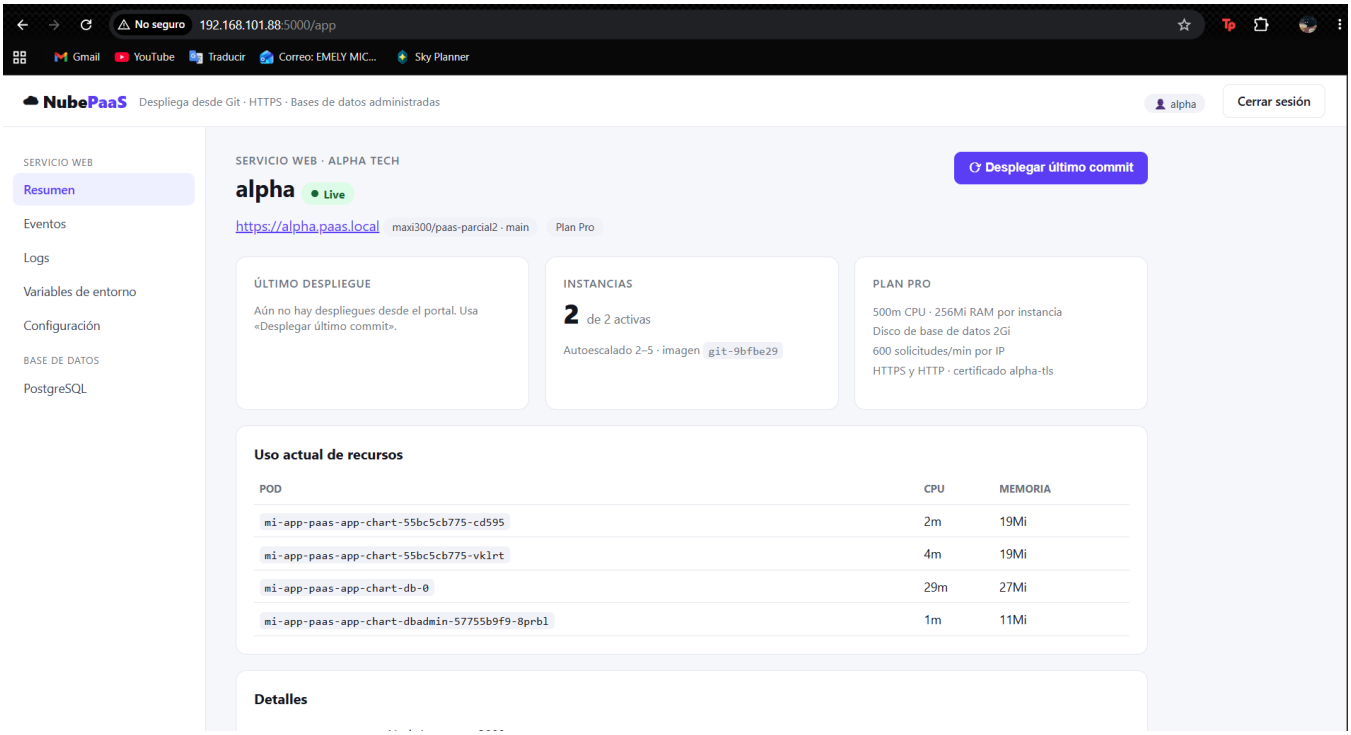


Figura 22. Página del servicio de Alpha: estado Live, URL, plan, último despliegue, instancias y uso de CPU/memoria por pod.

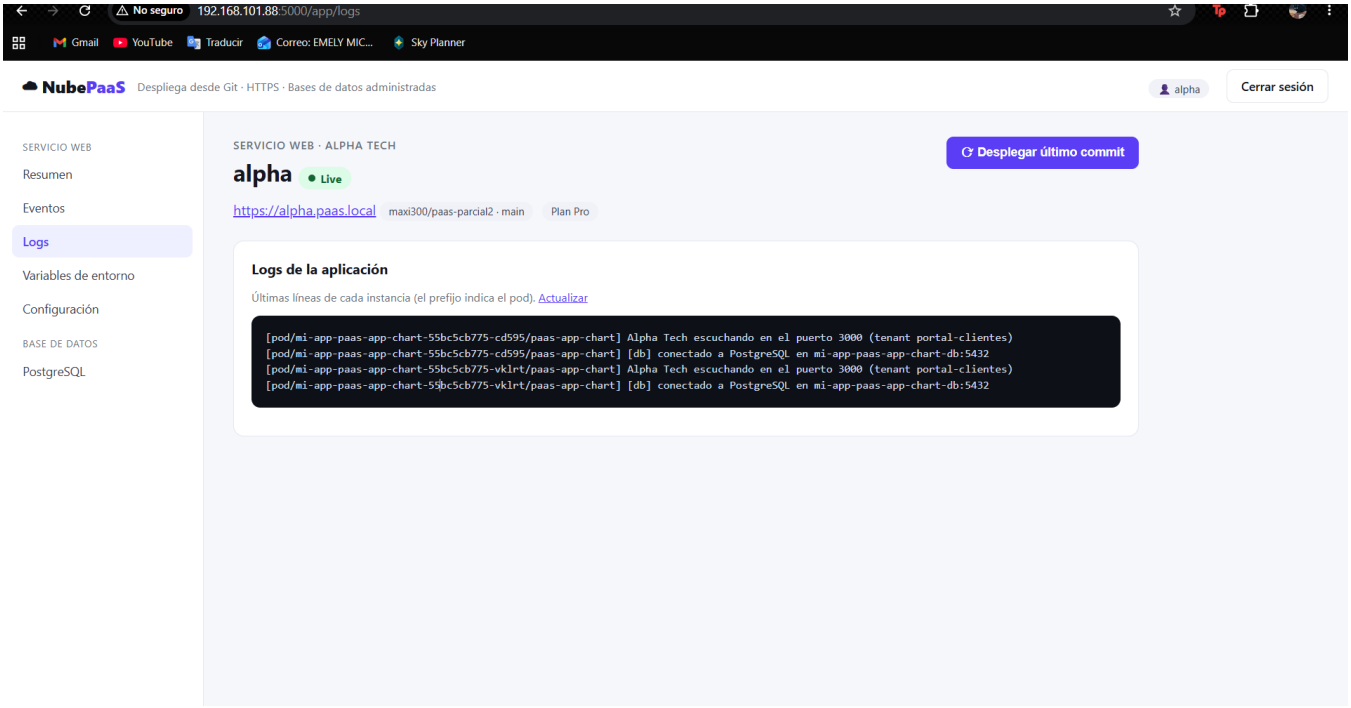


Figura 23. Logs de las instancias, con el pod como prefijo de cada línea.

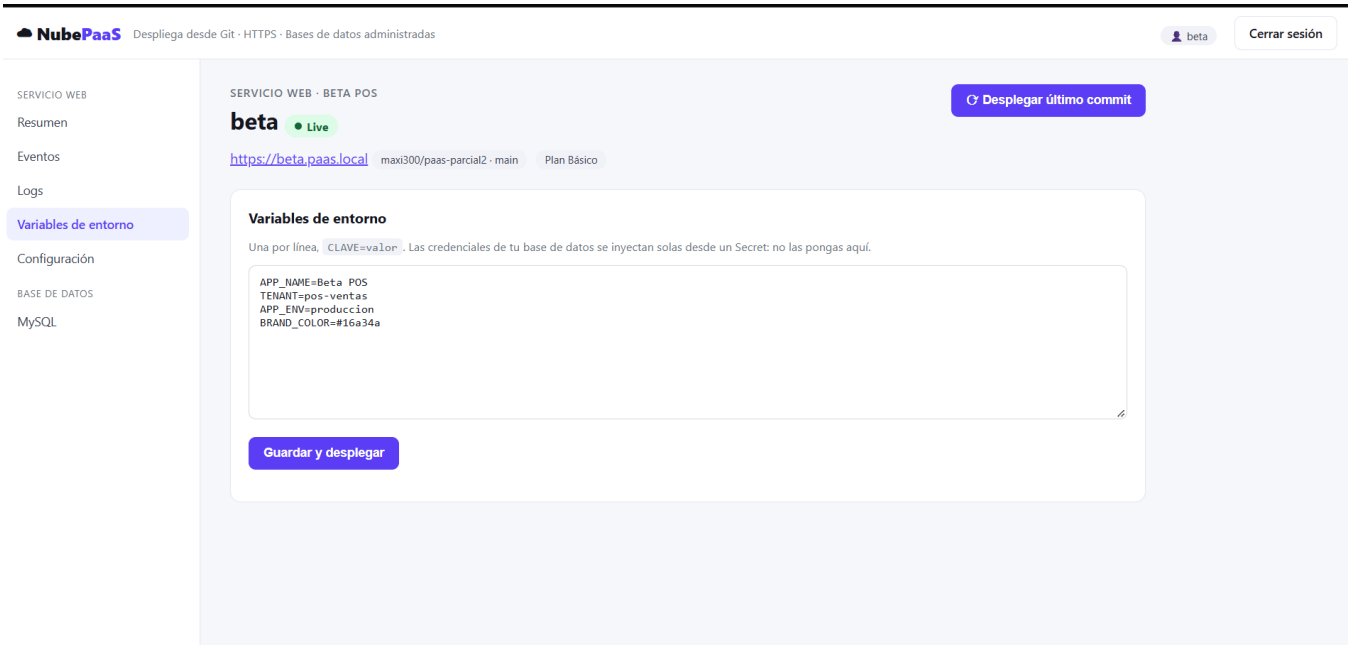


Figura 24. Variables de entorno editables por el cliente (las credenciales de la BD llegan desde un Secret).

9.3 Alta de un cliente nuevo desde el portal: TecnoMorazán

Con solo llenar el formulario (nombre `tecno-morazan`, repositorio, carpeta `apps/alpha-node`, Node.js, PostgreSQL, Plan Básico), la plataforma ejecutó en **235 s**: crear namespace → cuota del plan, LimitRange y NetworkPolicy → certificado SSL firmado por la CA → `git clone` → construir la imagen en los 2 nodos → `helm upgrade --install` → **Live**. El proceso completo está grabado en el video.

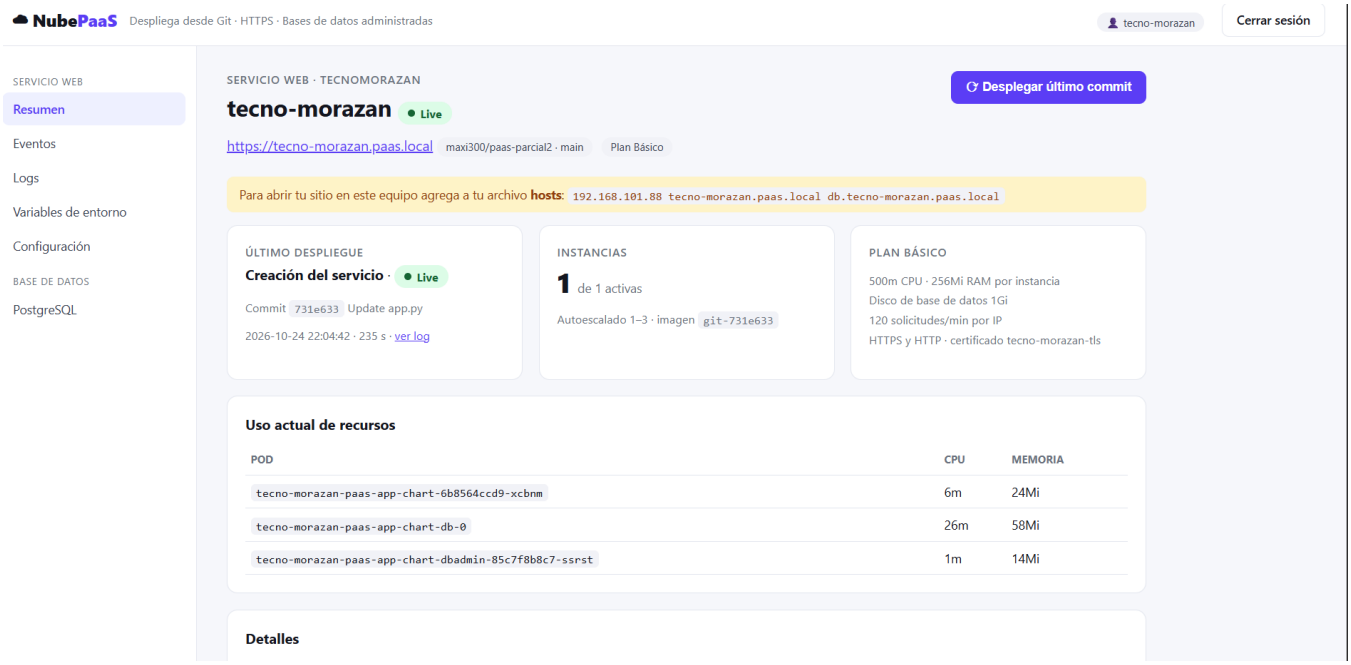


Figura 25. Servicio tecno-morazan en Live tras crearse desde el portal.

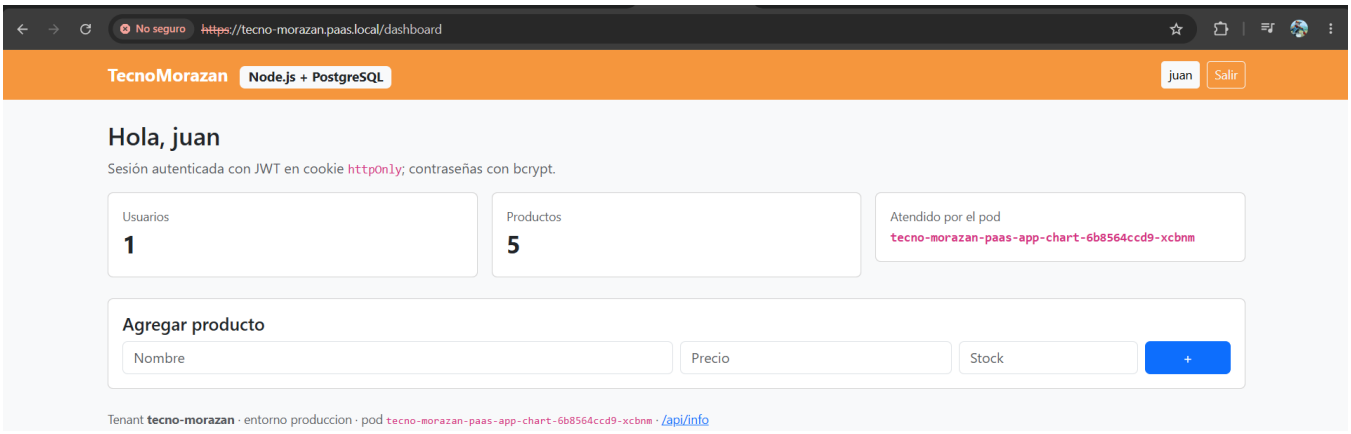


Figura 26. La tienda de TecnoMorazán funcionando por HTTPS con su color y su propia base PostgreSQL.

10. Despliegue desde Git (experiencia tipo Render)

Igual que en Render, el cliente **solo conecta su repositorio**: indica la URL, la rama y la carpeta con su Dockerfile. Cada vez que sube un cambio, presiona **«Desplegar último commit»** y la plataforma hace el resto. El diagrama resume los 8 pasos que ejecuta NubePaaS:



Figura 27. Flujo del botón «Desplegar último commit» en NubePaaS.

10.1 El código en GitHub

El repositorio `maxi300/paas-parcial2` se creó desde la VM y se subió con un token personal (que nunca se guardó en el repositorio). Antes del primer push se verificó que **no hubiera llaves** en los archivos versionados:

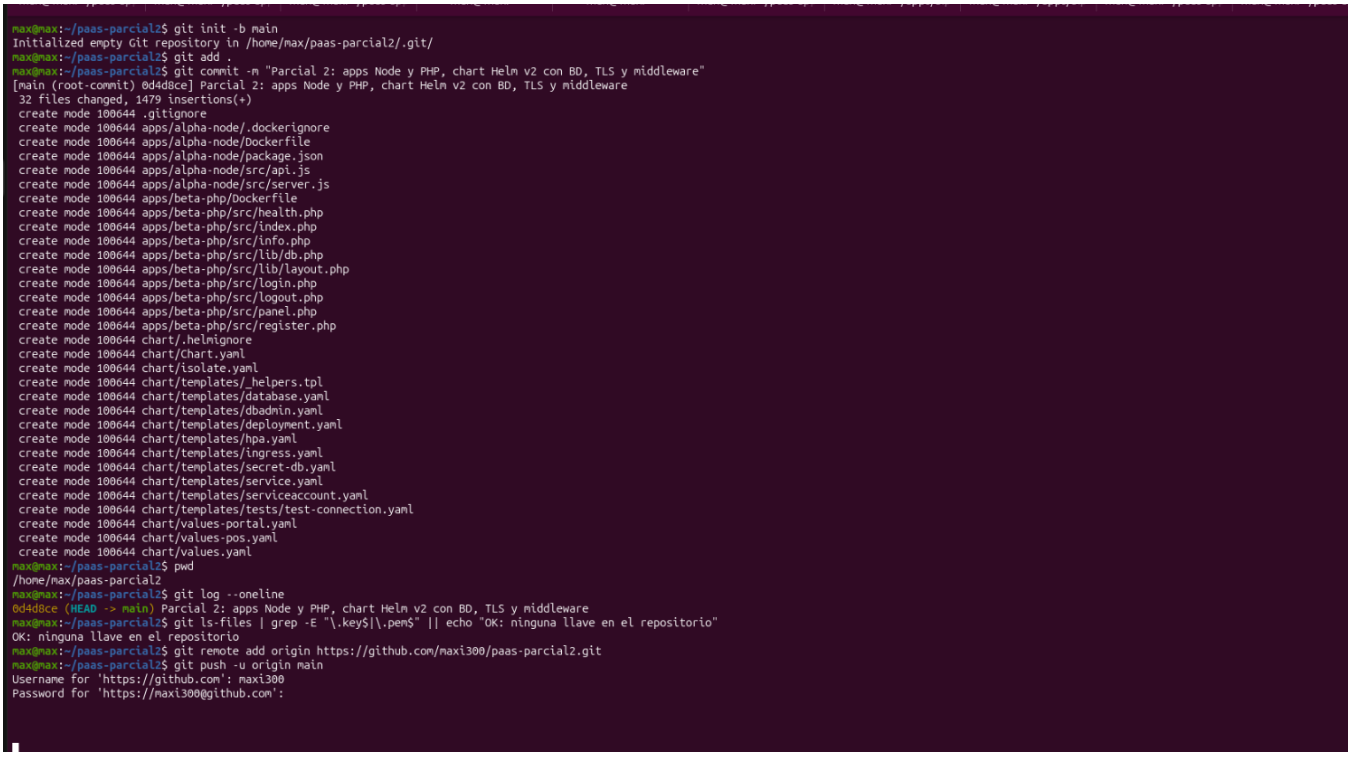


Figura 28. Terminal de la VM: git init, commit inicial con apps, chart y panel; comprobación «OK: ninguna llave en el repositorio» y git push.

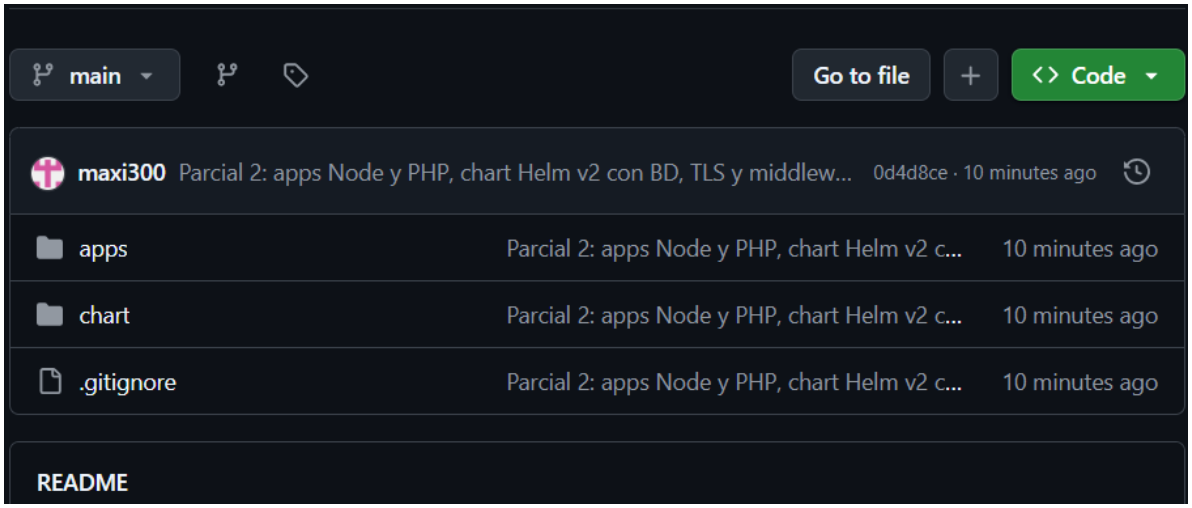


Figura 29. El repositorio en GitHub con las carpetas apps/, chart/ y .gitignore.

10.2 Primer despliegue desde Git (commit 9bfbe29)

Desde el panel se pidió desplegar el último commit de cada cliente. NubePaaS clonó la rama `main`, construyó la imagen `alpha-node-app:git-9bfbe29` con el Dockerfile de `apps/alpha-node` y la aplicó con Helm como **revisión 6**:



Figura 30. Trabajo «Alpha Tech · Desplegar desde Git» en curso: git clone del commit 9bfbe29 y construcción de la imagen en cada nodo.

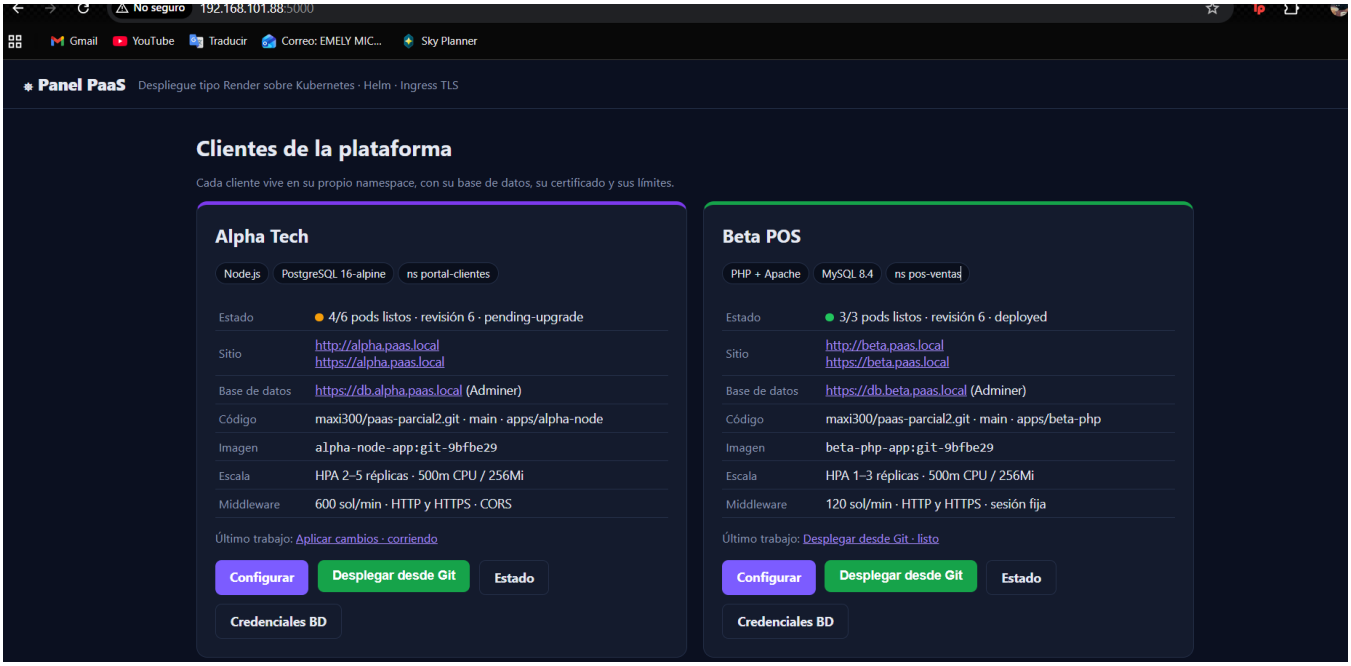


Figura 31. Mientras Helm aplica, el panel muestra «revisión 6 · pending-upgrade» y la imagen nueva alpha-node-app:git-9bfbe29.

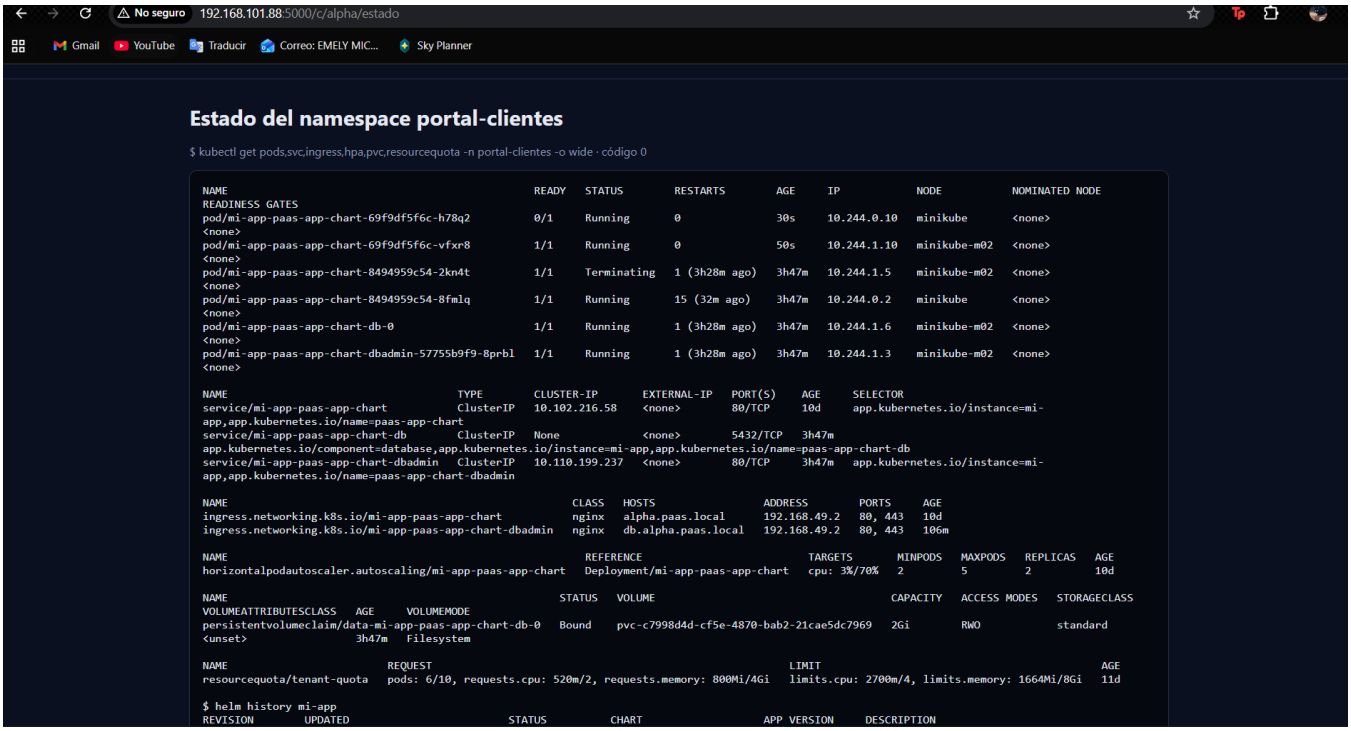


Figura 32. Estado del namespace durante el relevo: los pods nuevos arrancan mientras los viejos pasan a Terminating; no hay caída.

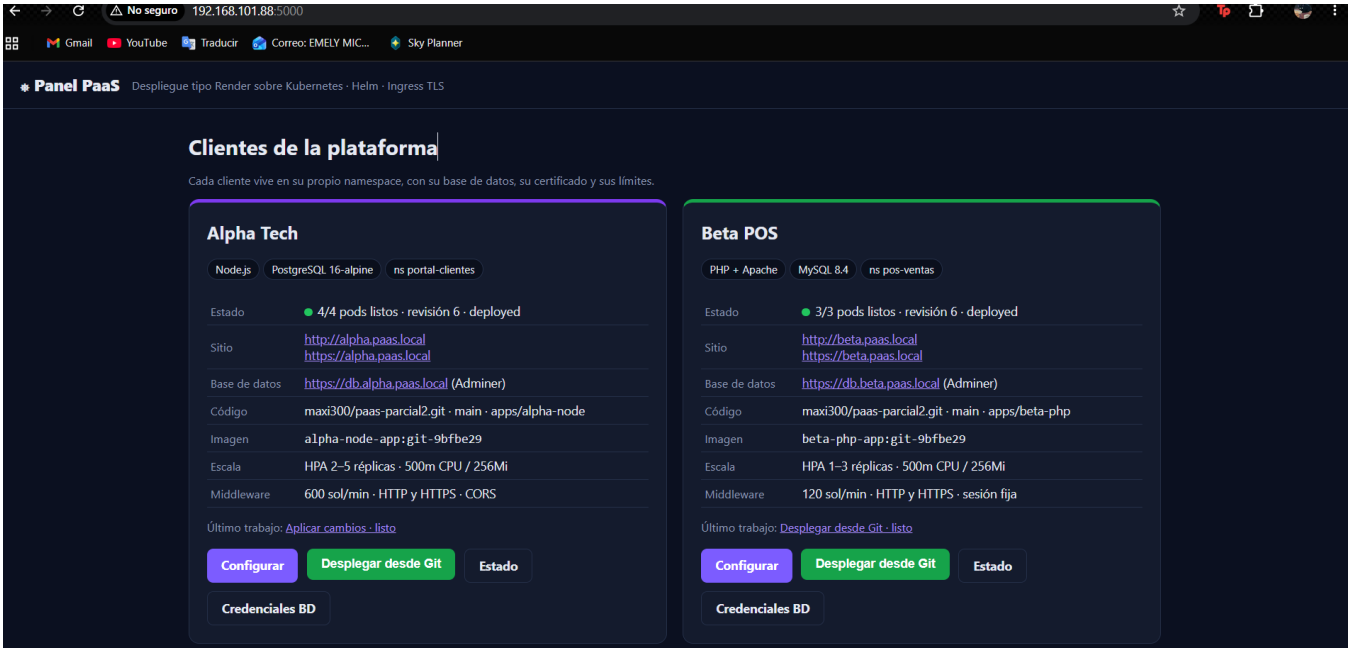


Figura 33. Terminado: los dos clientes en «revisión 6 · deployed», cada uno con su imagen git-9bfbe29.

10.3 Un cambio real en el código (commit 1746196)

Para demostrar el ciclo completo se modificó `apps/alpha-node/src/server.js` en GitHub agregando el panel **«Productos · CRUD con la API REST»** (commit `1746196`: «Alpha: panel CRUD que usa la API REST»). El cliente entró a su portal y presionó «Desplegar último commit»:

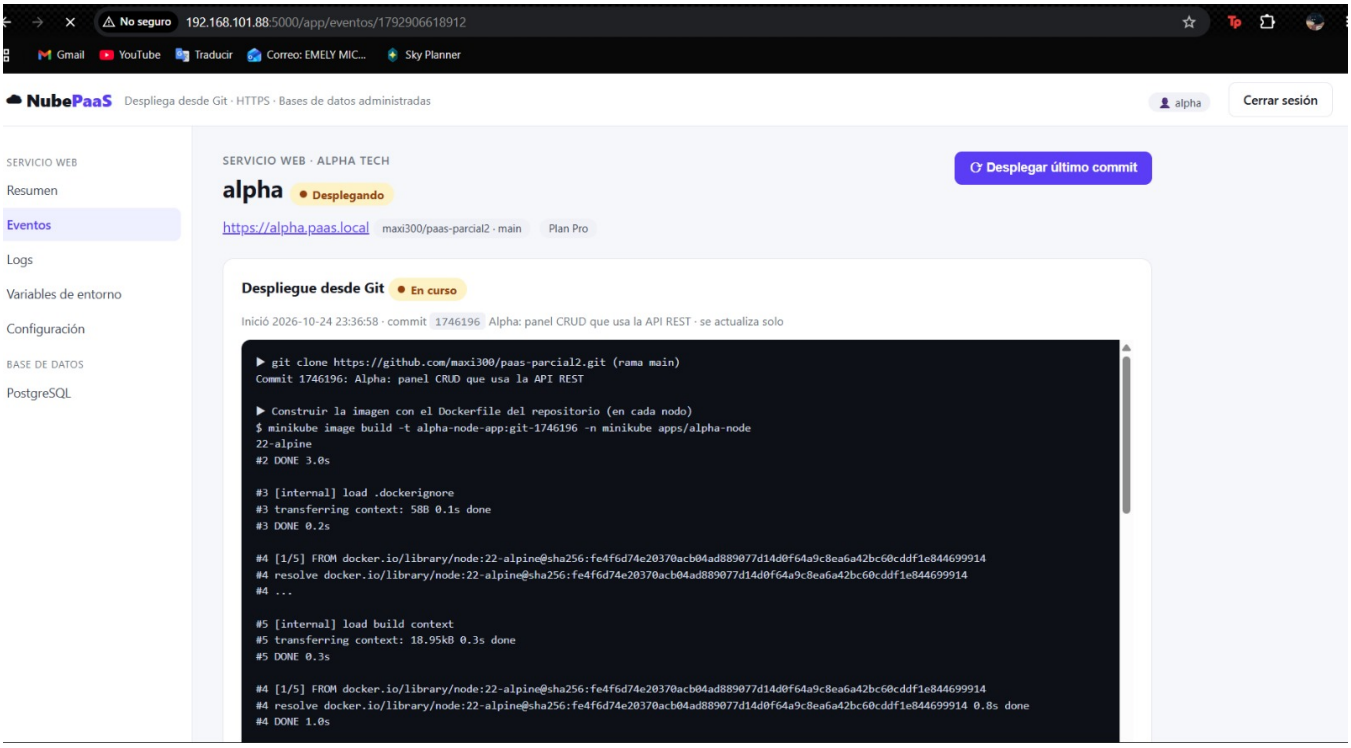


Figura 34. Portal del cliente → Eventos: log en vivo del despliegue del commit 1746196 (git clone y construcción de la imagen con node:22-alpine).

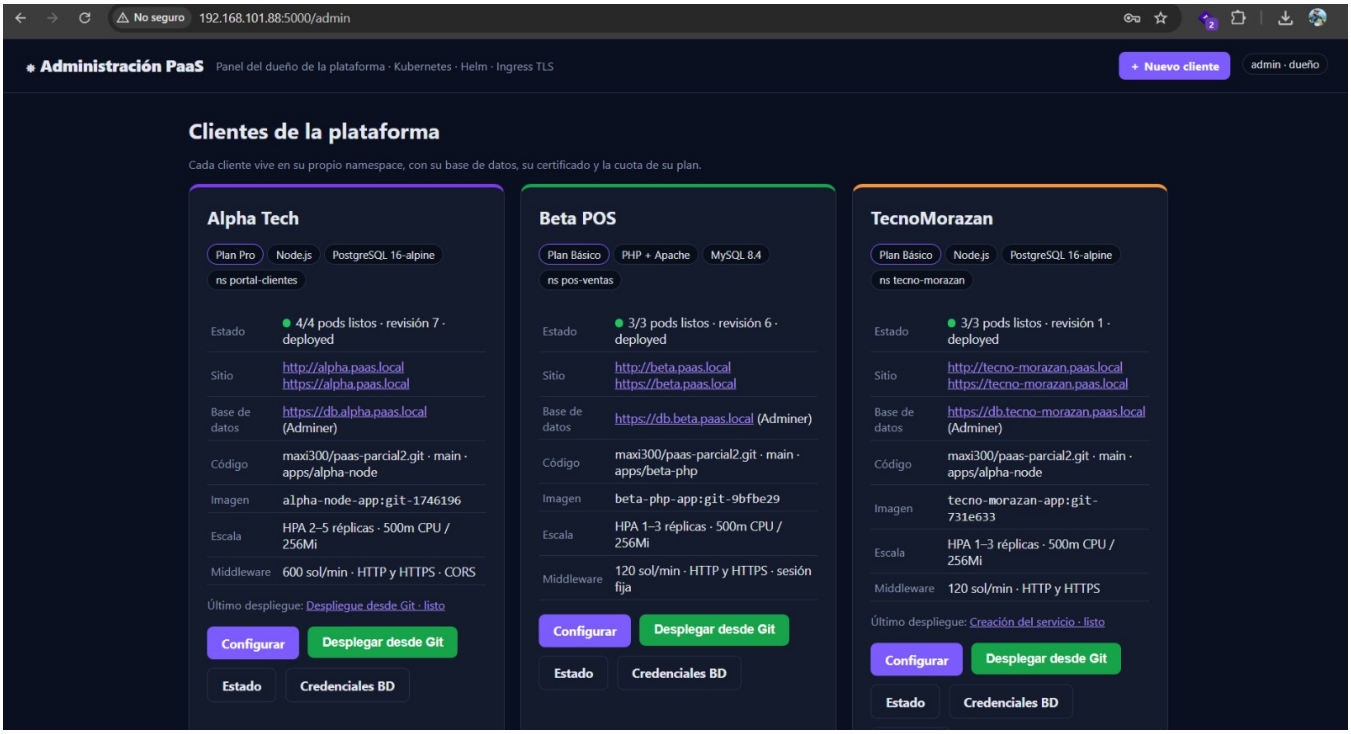


Figura 35. Panel del dueño al terminar: Alpha Tech en «revisión 7 · deployed» con la imagen alpha-node-app:git-1746196.

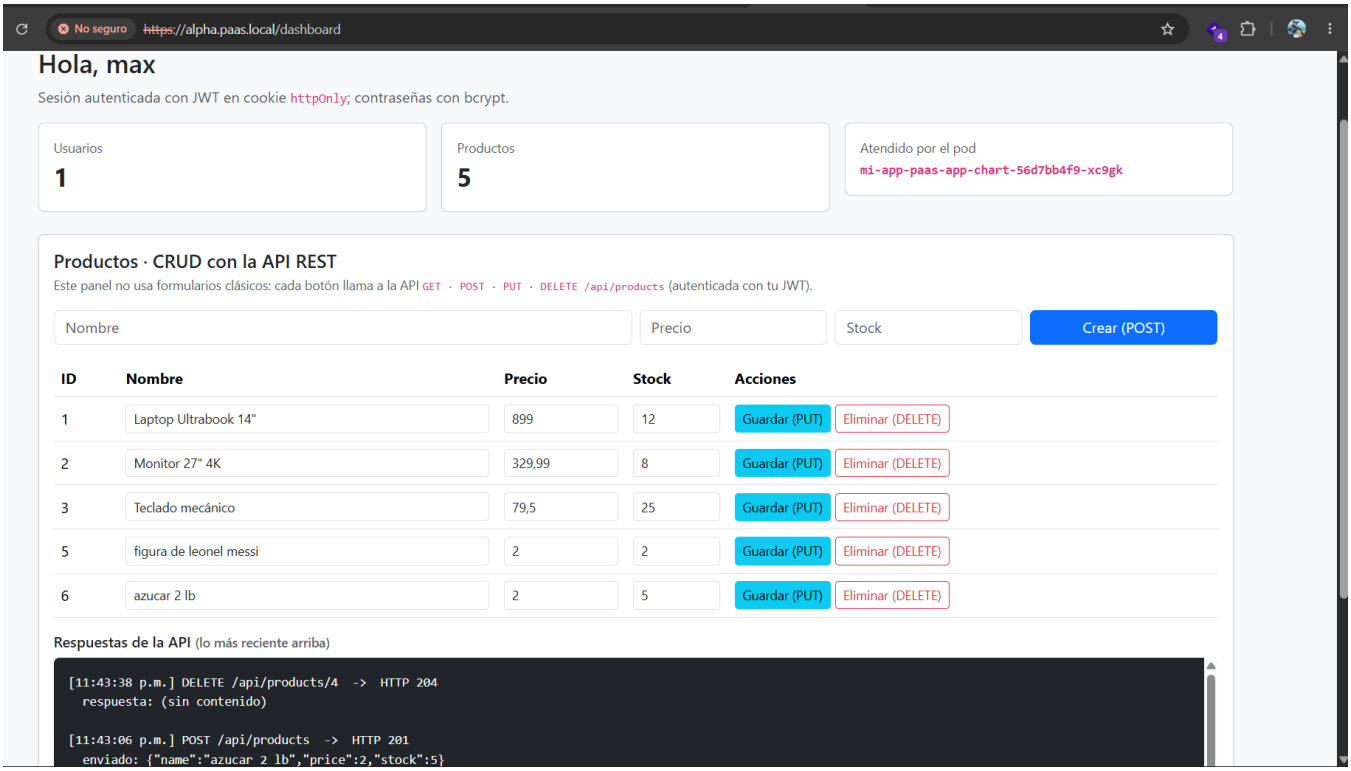


Figura 36. Resultado en https://alpha.paas.local/dashboard: el panel nuevo, construido desde el commit, consumiendo la API REST.

Resumen del ciclo: commit en GitHub → «Desplegar último commit» → imagen `git-1746196` construida en los 2 nodos → `helm upgrade` (revisión 7) → relevo de pods sin caída → la app muestra el cambio. El cliente nunca usó la terminal ni Kubernetes.

11. Wappalyzer

Wappalyzer detecta las tecnologías del lado del cliente. Se nota la diferencia de stacks entre clientes. Las bases de datos no son visibles desde el navegador porque solo son accesibles dentro de su namespace; su versión se verifica en `/api/info` y `/info.php` (sección 4).

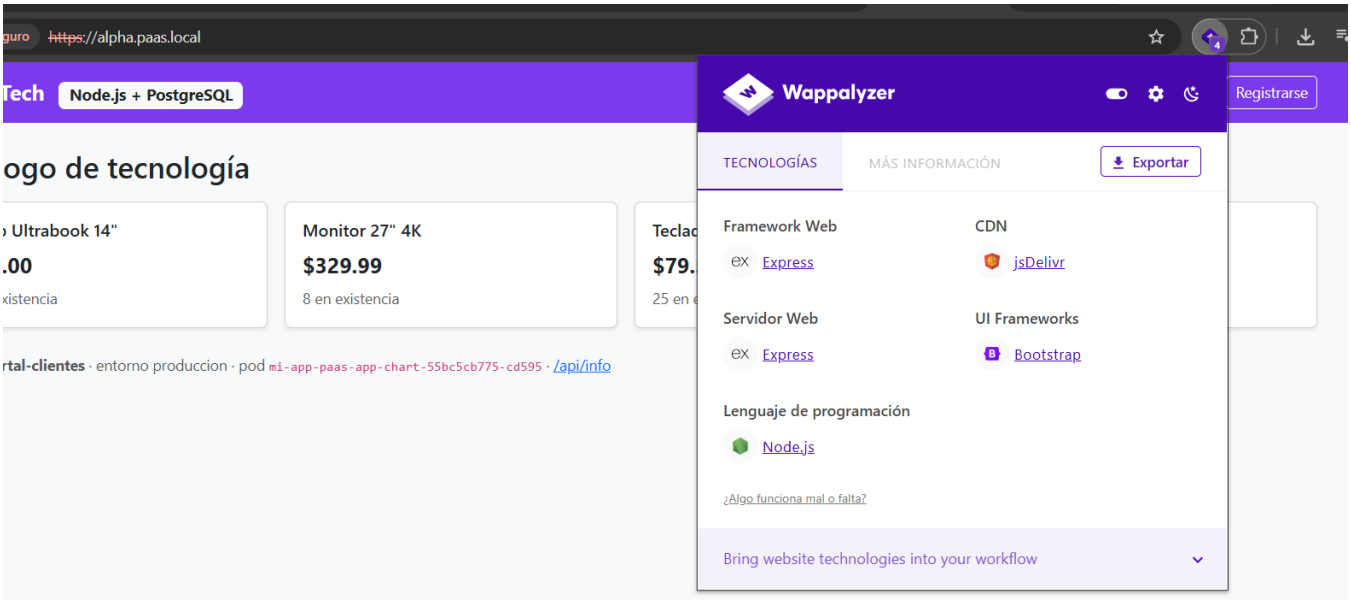


Figura 37. Cliente 1 (Alpha Tech): Express, Node.js, Bootstrap y jsDelivr.

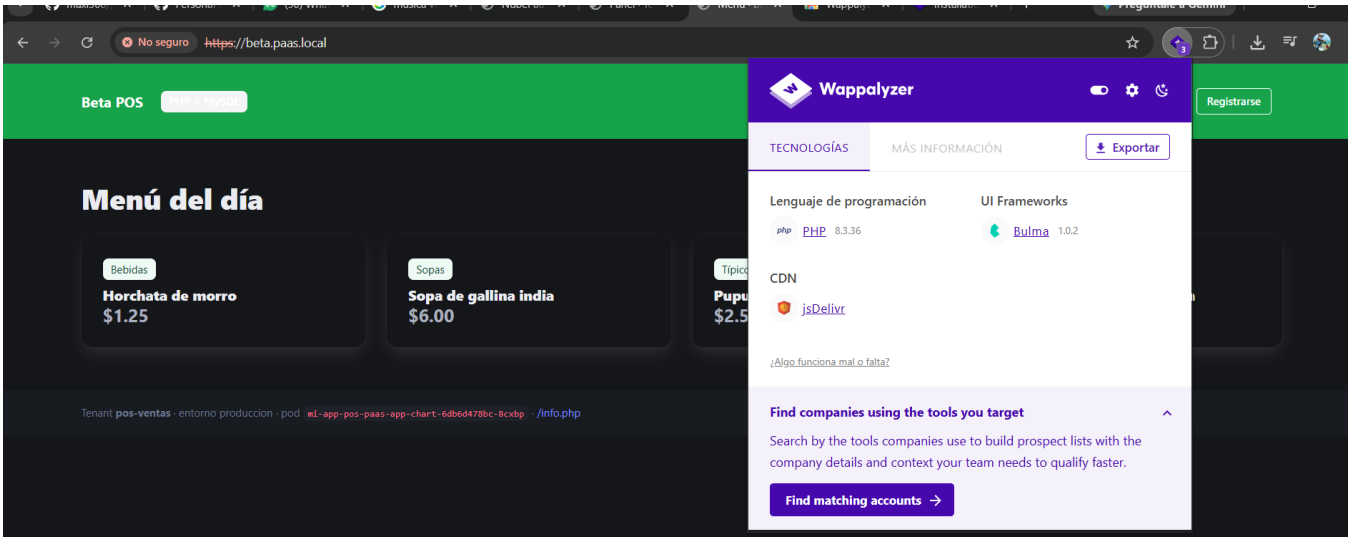


Figura 38. Cliente 2 (Beta POS): PHP 8.3.36, Bulma 1.0.2 y jsDelivr: un stack distinto.

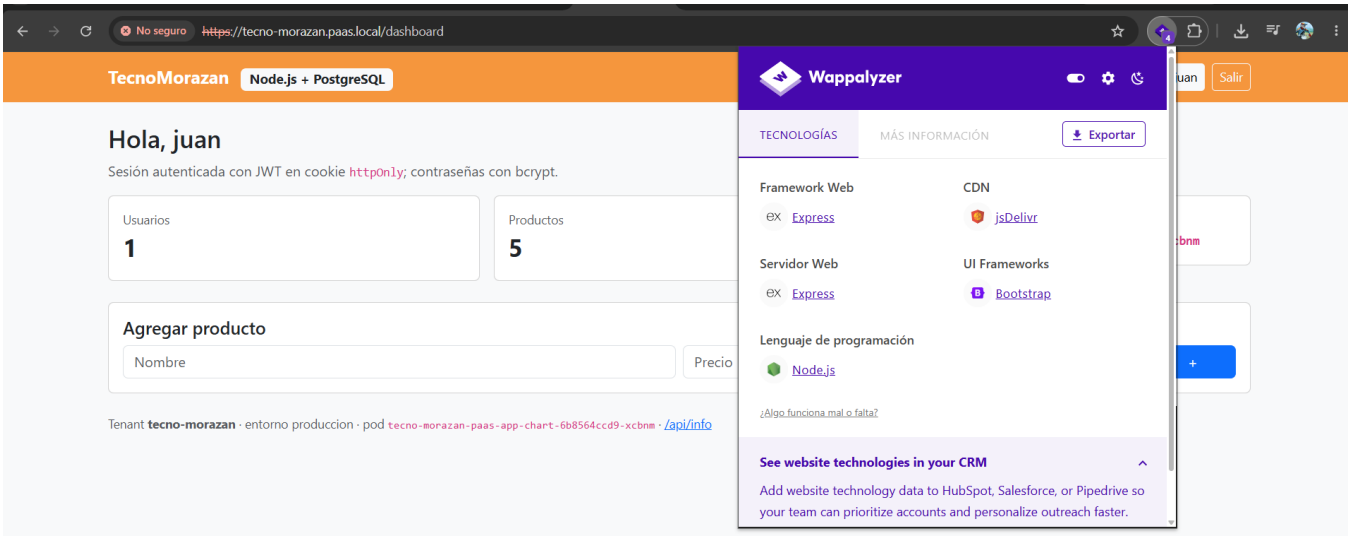


Figura 39. TecnoMorazán (creado desde el portal con el mismo stack que Alpha).

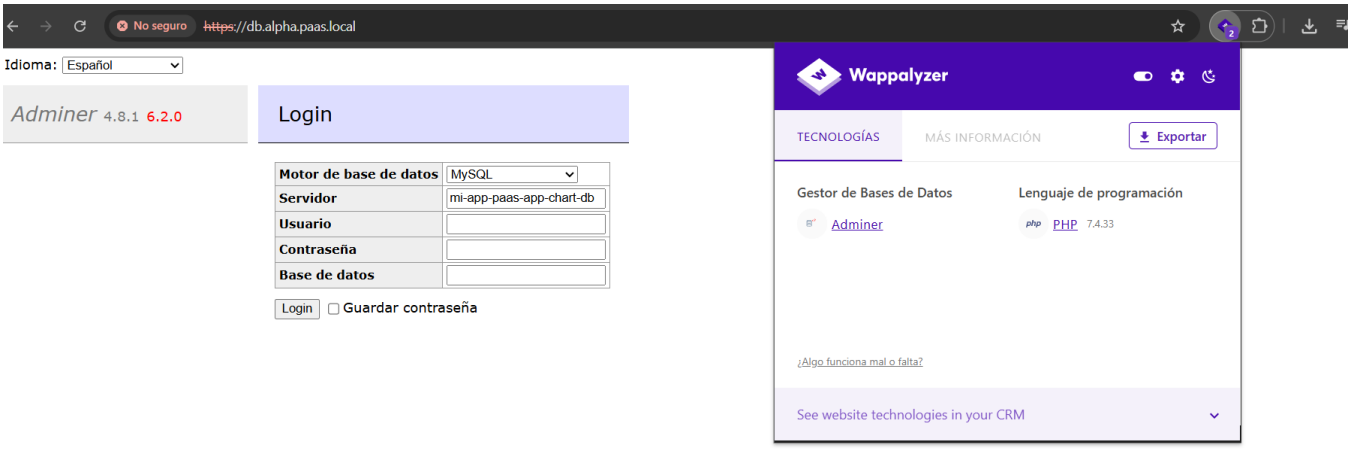


Figura 40. Adminer de la base de datos: Adminer + PHP.

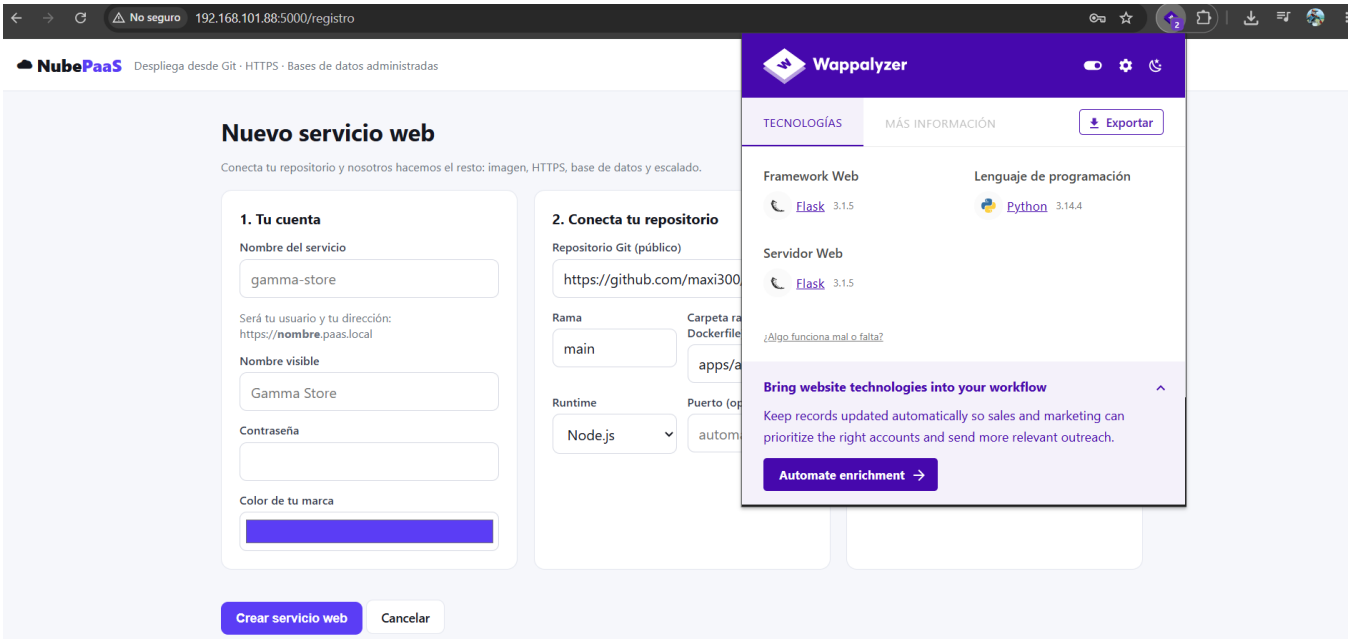


Figura 41. La propia plataforma NubePaaS: Flask 3.1.5 + Python 3.14.

12. Pruebas funcionales

12.1 CRUD en PHP (Beta POS)

Con un usuario registrado se crearon, editaron y eliminaron platillos; el menú público refleja cada cambio guardado en MySQL.

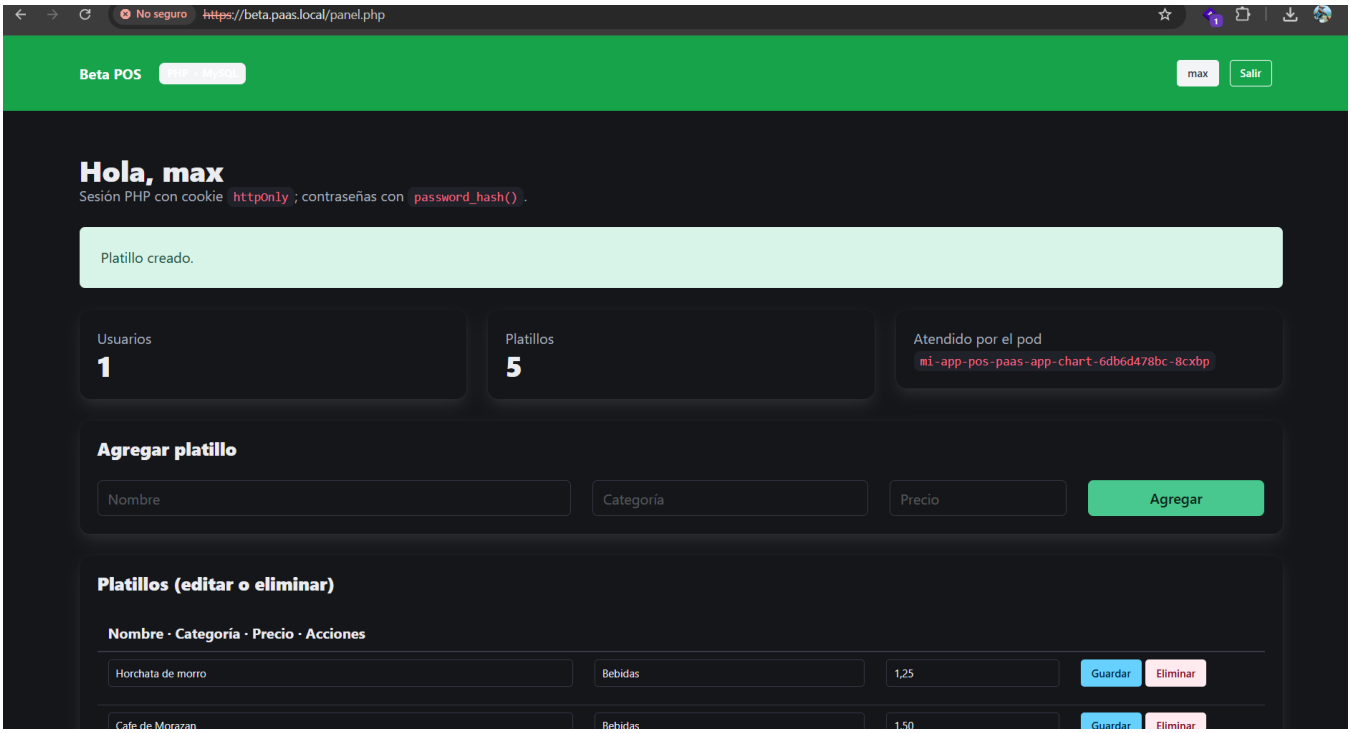


Figura 42. Crear: se agrega el platillo «Café de Morazán».

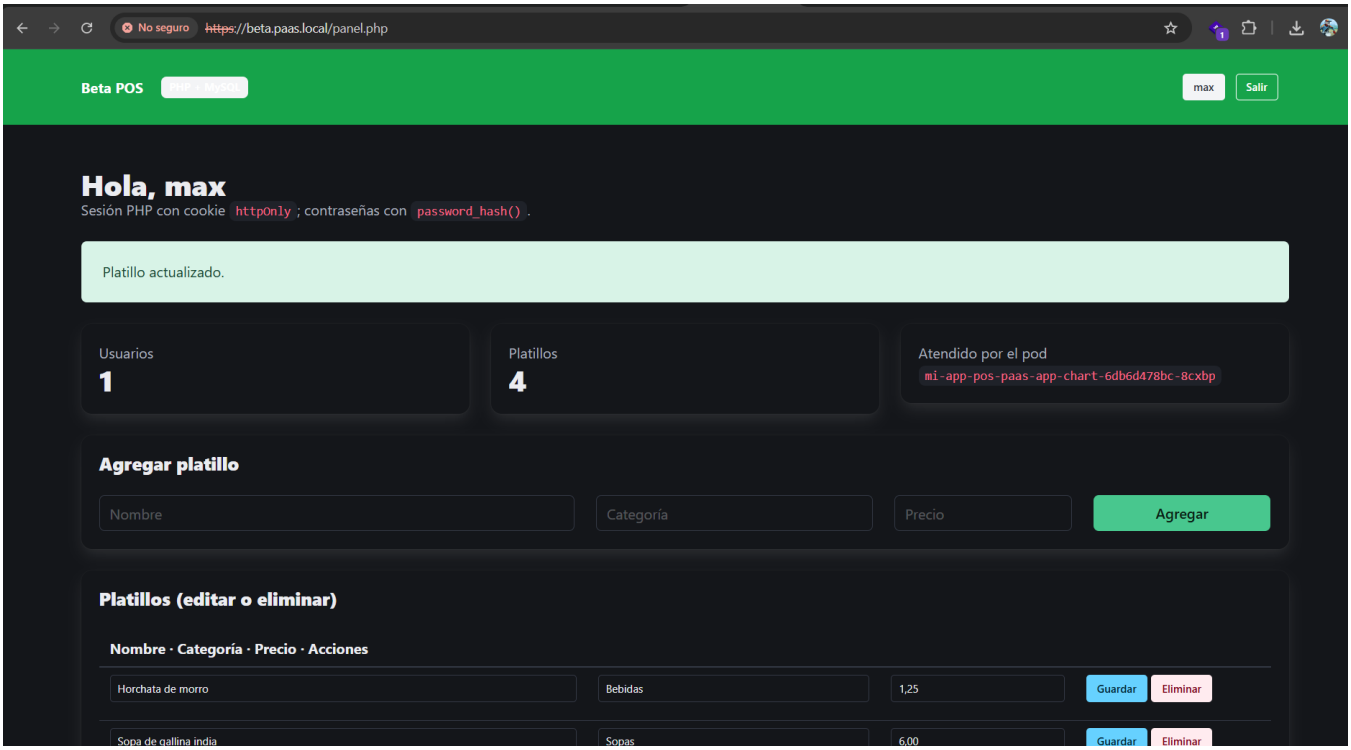


Figura 43. Editar: se cambia el precio de las pupusas y aparece «Platillo actualizado».

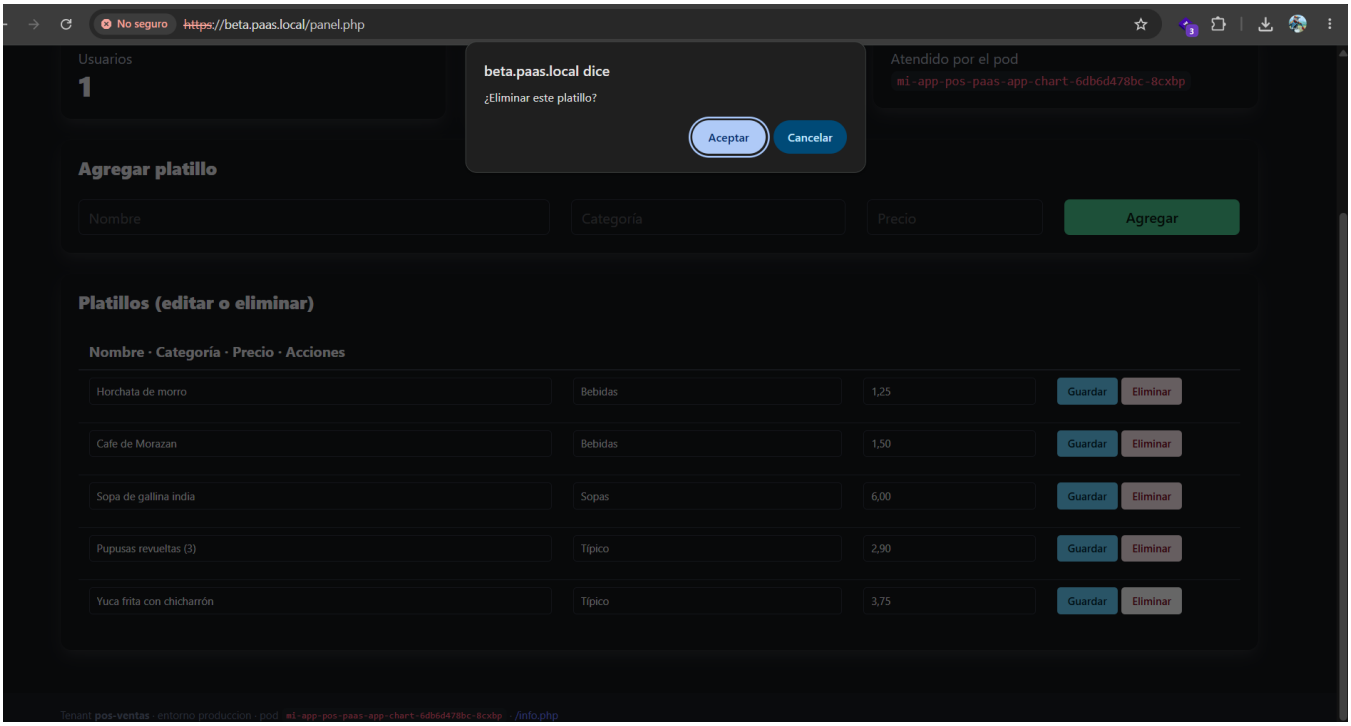


Figura 44. Eliminar: confirmación antes de borrar la yuca frita.

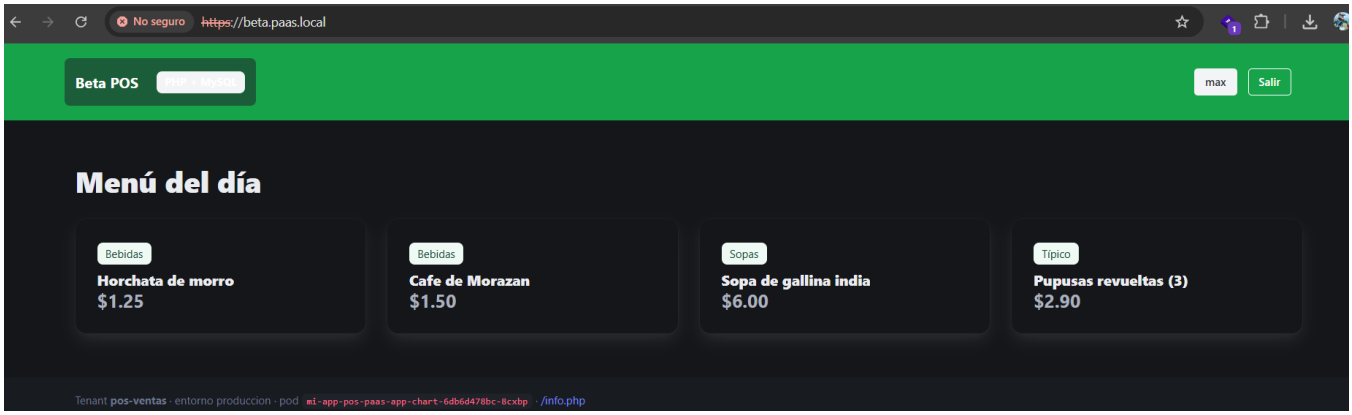


Figura 45. Menú público después de las operaciones: aparece el café, las pupusas en $2.90 y ya no está la yuca.

12.2 API REST en Node (Alpha Tech)

El panel de productos de Alpha no usa formularios clásicos: cada botón llama a la API REST autenticada con el JWT de la sesión, y la caja inferior muestra cada petición con su respuesta.

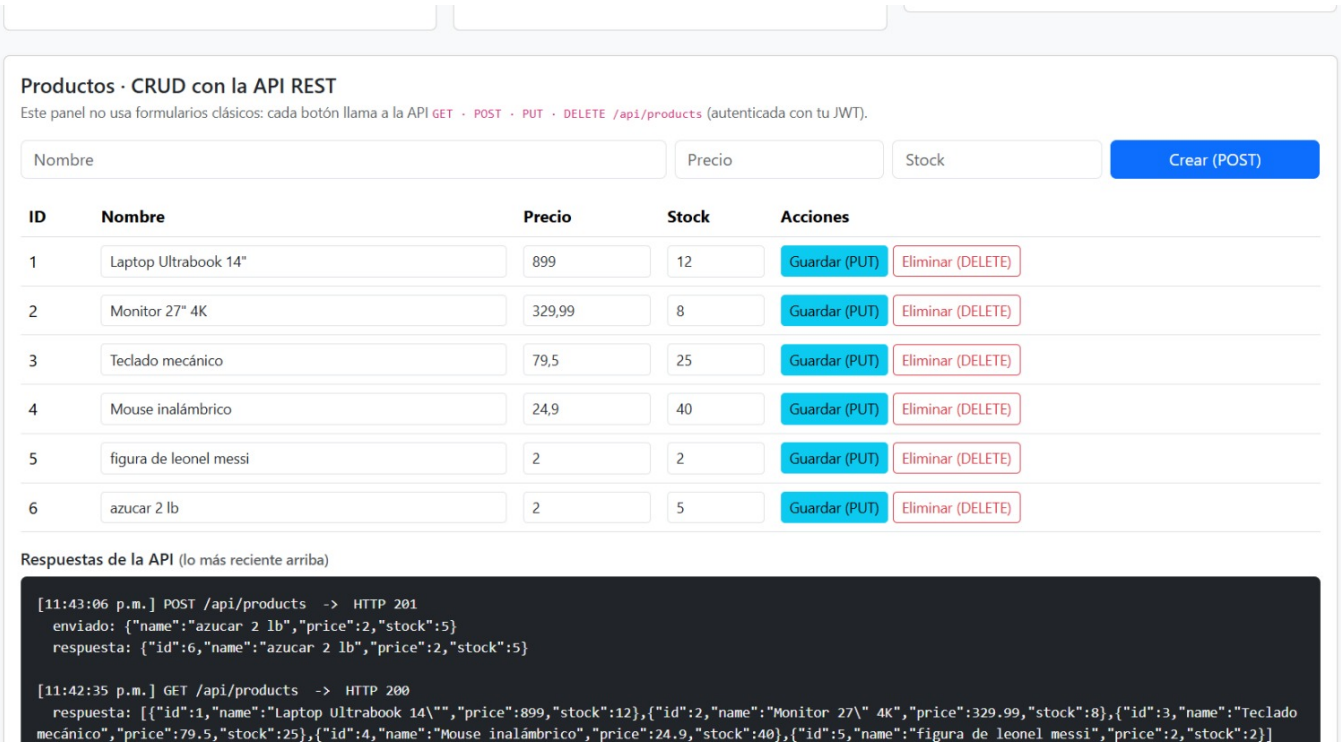


Figura 46. Panel «Productos · CRUD con la API REST» con la caja de respuestas: POST 201 (azúcar 2 lb) y GET 200.

```
[11:43:38 p.m.] DELETE /api/products/4  ->  HTTP 204   respuesta: (sin contenido)
[11:43:06 p.m.] POST /api/products  ->  HTTP 201
  enviado: {"name":"azucar 2 lb","price":2,"stock":5}
  respuesta: {"id":6,"name":"azucar 2 lb","price":2,"stock":5}
[11:42:35 p.m.] GET /api/products  ->  HTTP 200
  respuesta: [{"id":1,"name":"Laptop Ultrabook 14\"","price":899,"stock":12}, … ,
{"id":5,"name":"figura de leonel messi","price":2,"stock":2}]
Sin token:  POST /api/products  ->  HTTP 401  {"error":"Token inválido o ausente"}
```

12.3 Aislamiento entre clientes

Desde un pod temporal dentro del namespace de Alpha se intentó conectar a su propia base y a la base de Beta:

```
$ kubectl run prueba-red -n portal-clientes --rm -it --restart=Never --image=busybox:1.36 -- sh
-c '...'
== Alpha -> su PostgreSQL:
mi-app-paas-app-chart-db (10.244.1.6:5432) open
PERMITIDO
== Alpha -> MySQL de Beta:
nc: mi-app-pos-paas-app-chart-db.pos-ventas.svc.cluster.local (10.244.1.9:3306): Connection
timed out
BLOQUEADO por la NetworkPolicy
pod "prueba-red" deleted from portal-clientes namespace
```

Aunque el pod de Alpha conoce la IP y el puerto de la base de Beta, la conexión expira: la NetworkPolicy `isolate-namespace` solo acepta tráfico del mismo namespace, del Ingress y de monitoreo. En la plataforma, un cliente tampoco puede abrir la página de otro (HTTP 403) y cada uno tiene su propia cuota (ResourceQuota) y valores por defecto (LimitRange).

13. Autoescalado (HPA): demostración en vivo

Cada cliente tiene un **HorizontalPodAutoscaler** según su plan: Alpha (Pro) puede crecer de **2 a 5** réplicas y Beta (Básico) de **1 a 3**. El HPA mide el uso de CPU de los pods contra lo que piden (`requests.cpu: 100m`) y agrega réplicas cuando el promedio supera el objetivo.

En reposo Alpha usa apenas **1–3 %** de su CPU, muy por debajo del objetivo normal de 70 %. Para que el escalado se viera en la demostración, desde el panel del dueño se bajó el objetivo a **10 %** (el mínimo que permite el formulario) y se lanzó la prueba de carga con más usuarios simultáneos:

```
# Panel del dueño → Alpha Tech → Configurar → Escalado: Mín 2 · Máx 5 · CPU % = 10 → Guardar y
aplicar (helm upgrade)
$ kubectl get hpa -n portal-clientes -w            # terminal 1: observar el HPA
$ ~/paas-parcial2/pruebas/carga.sh 180 10          # terminal 2: 10 procesos por cliente durante
3 minutos
```



Figura 47. El cambio del objetivo de CPU se hizo desde el panel (izq.) y se aplicó con Helm como cualquier otra configuración (der.).

La terminal del `-w` muestra cada decisión del HPA: al aplicarse el cambio el objetivo pasa de `70%` a `10%`; con la carga la CPU sube a **15–18 %**, el HPA calcula que necesita más pods y pasa de **2 a 4** réplicas y luego a **5**, el máximo del plan. Al terminar la carga la CPU vuelve a 2–4 %.

```
max@max:~/apps/alpha-node$ kubectl get hpa -n portal-clientes -w
NAME                   REFERENCE                         TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 1%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 3%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 1%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 3%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 4%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 4%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 6%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 7%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 15%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 18%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 18%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 18%/10%   2         5         4          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 10%/10%   2         5         4          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 5%/10%    2         5         4          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 3%/10%    2         5         4          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 17%/10%   2         5         4          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 17%/10%   2         5         5          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/10%    2         5         5          11d


mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 3%/10%    2         5         5          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 4%/10%    2         5         5          11d
   kubectl get pods -n portal-clientes -o widemi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/10%    2         5         5          11d
^Cmax@max:~/apps/alpha-node   kubectl get pods -n portal-clientes -o widede
NAME                                      READY   STATUS    RESTARTS      AGE     IP            NODE           NOMINATED NODE   READINESS GATES
mi-app-paas-app-chart-77578c7985-97r8z    1/1     Running   0             8m53s   10.244.1.22   minikube-m02   <none>           <none>
mi-app-paas-app-chart-77578c7985-bqswb    1/1     Running   0             14m     10.244.1.21   minikube-m02   <none>           <none>
mi-app-paas-app-chart-77578c7985-cwdz7    1/1     Running   1             8m52s   10.244.0.17   minikube       <none>           <none>
mi-app-paas-app-chart-77578c7985-nqtjv    1/1     Running   0             4m58s   10.244.1.23   minikube-m02   <none>           <none>
mi-app-paas-app-chart-77578c7985-s6h8s    1/1     Running   1             14m     10.244.0.16   minikube       <none>           <none>
mi-app-paas-app-chart-db-0                1/1     Running   1 (18h ago)   18h     10.244.1.6    minikube-m02   <none>           <none>
mi-app-paas-app-chart-dbadmin-57755b9f9-8prbl   1/1     Running   1 (18h ago)   18h     10.244.1.3    minikube-m02   <none>           <none>
max@max:~/apps/alpha-node$ ^[[200~kubectl get hpa -n portal-clientes -w
kubectl: command not found
max@max:~/apps/alpha-node$ kubectl get hpa -n portal-clientes -w
NAME                   REFERENCE                         TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 2%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 4%/10%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 4%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 3%/70%   2         5         2          11d
mi-app-paas-app-chart   Deployment/mi-app-paas-app-chart   cpu: 3%/70%   2         5         2          11d
```

Figura 48. kubectl get hpa -w: TARGETS sube de 2 %/10 % a 18 %/10 % y REPLICAS pasa de 2 → 4 → 5; abajo, ya sin carga, vuelve a 2 réplicas y el objetivo regresa a 70 %.

```
^Cmax@max:~/apps/alpha-node   kubectl get pods -n portal-clientes -o widede
NAME                                      READY   STATUS    RESTARTS      AGE     IP            NODE           NOMINATED NODE   READINESS GATES
mi-app-paas-app-chart-77578c7985-97r8z    1/1     Running   0             8m53s   10.244.1.22   minikube-m02   <none>           <none>
mi-app-paas-app-chart-77578c7985-bqswb    1/1     Running   0             14m     10.244.1.21   minikube-m02   <none>           <none>
mi-app-paas-app-chart-77578c7985-cwdz7    1/1     Running   1             8m52s   10.244.0.17   minikube       <none>           <none>
mi-app-paas-app-chart-77578c7985-nqtjv    1/1     Running   0             4m58s   10.244.1.23   minikube-m02   <none>           <none>
mi-app-paas-app-chart-77578c7985-s6h8s    1/1     Running   1             14m     10.244.0.16   minikube       <none>           <none>
mi-app-paas-app-chart-db-0                1/1     Running   1 (18h ago)   18h     10.244.1.6    minikube-m02   <none>           <none>
mi-app-paas-app-chart-dbadmin-57755b9f9-8prbl   1/1     Running   1 (18h ago)   18h     10.244.1.3    minikube-m02   <none>           <none>
max@max:~/apps/alpha-node$
```

Figura 49. Los 5 pods de Alpha en Running, repartidos por Kubernetes entre los dos nodos (3 en minikube-m02 y 2 en minikube).

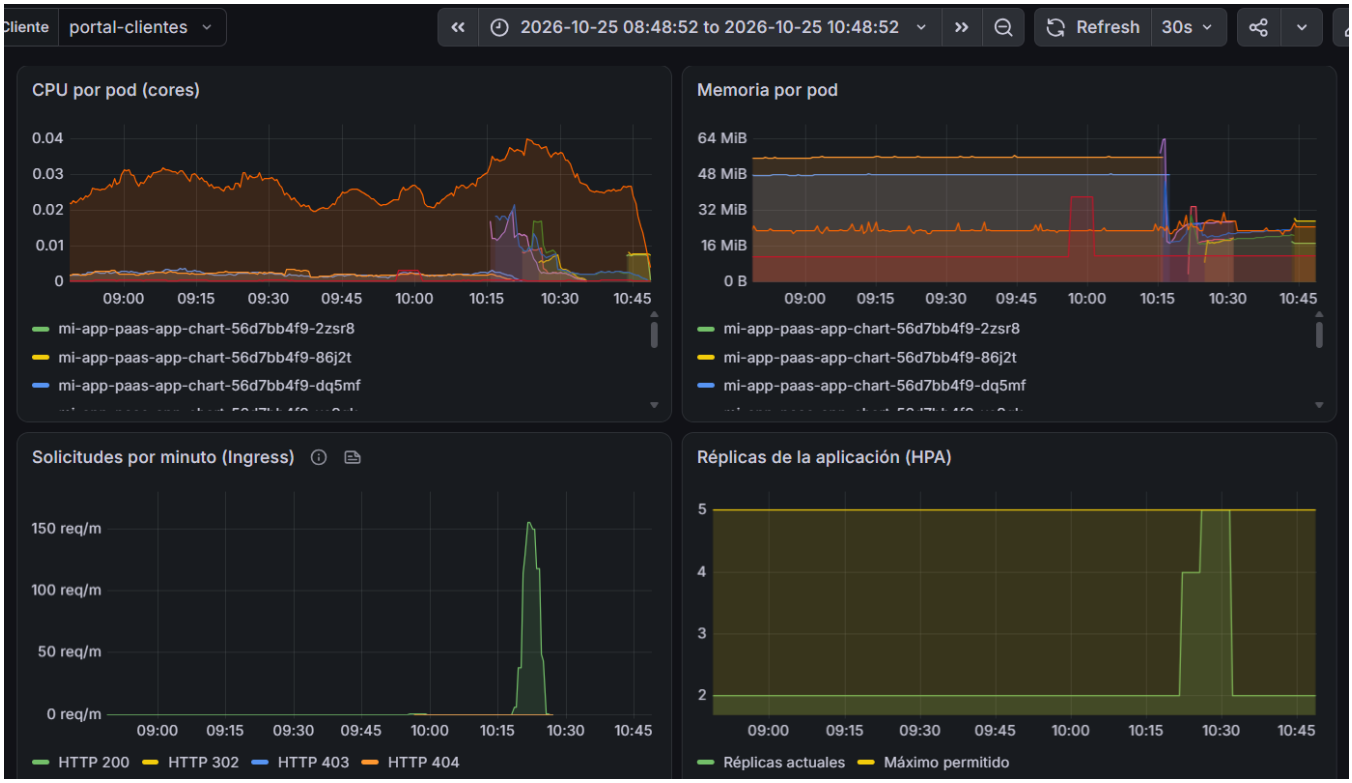


Figura 50. Grafana · PaaS Parcial 2 (portal-clientes): pico de ~155 solicitudes/min y el ciclo completo del HPA: 2 → 4 → 5 réplicas y de regreso a 2 (≈ 10:31).

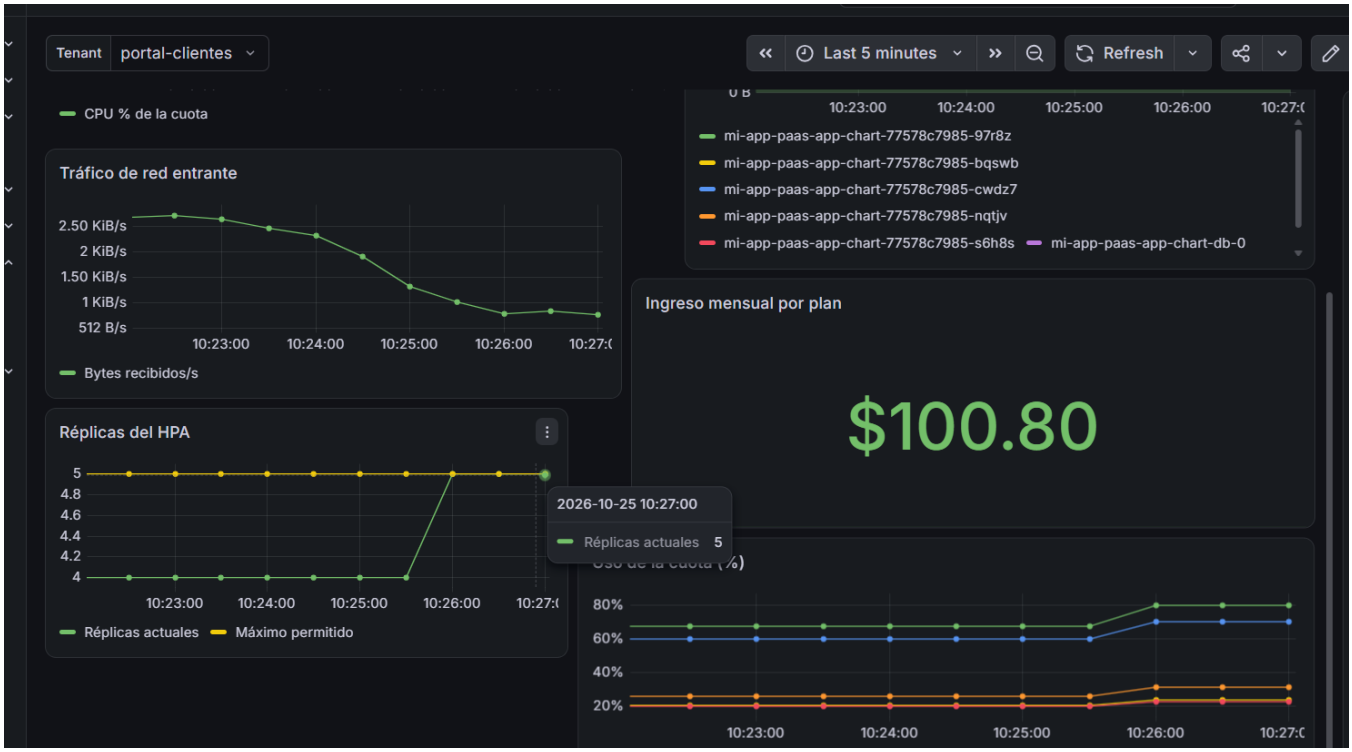


Figura 51. Grafana · PaaS por Tenant (últimos 5 min): réplicas 2 → 4 → 5 y la cuota consumida subiendo (limits.cpu de 40 % a 80 %, pods de 40 % a 70 %).

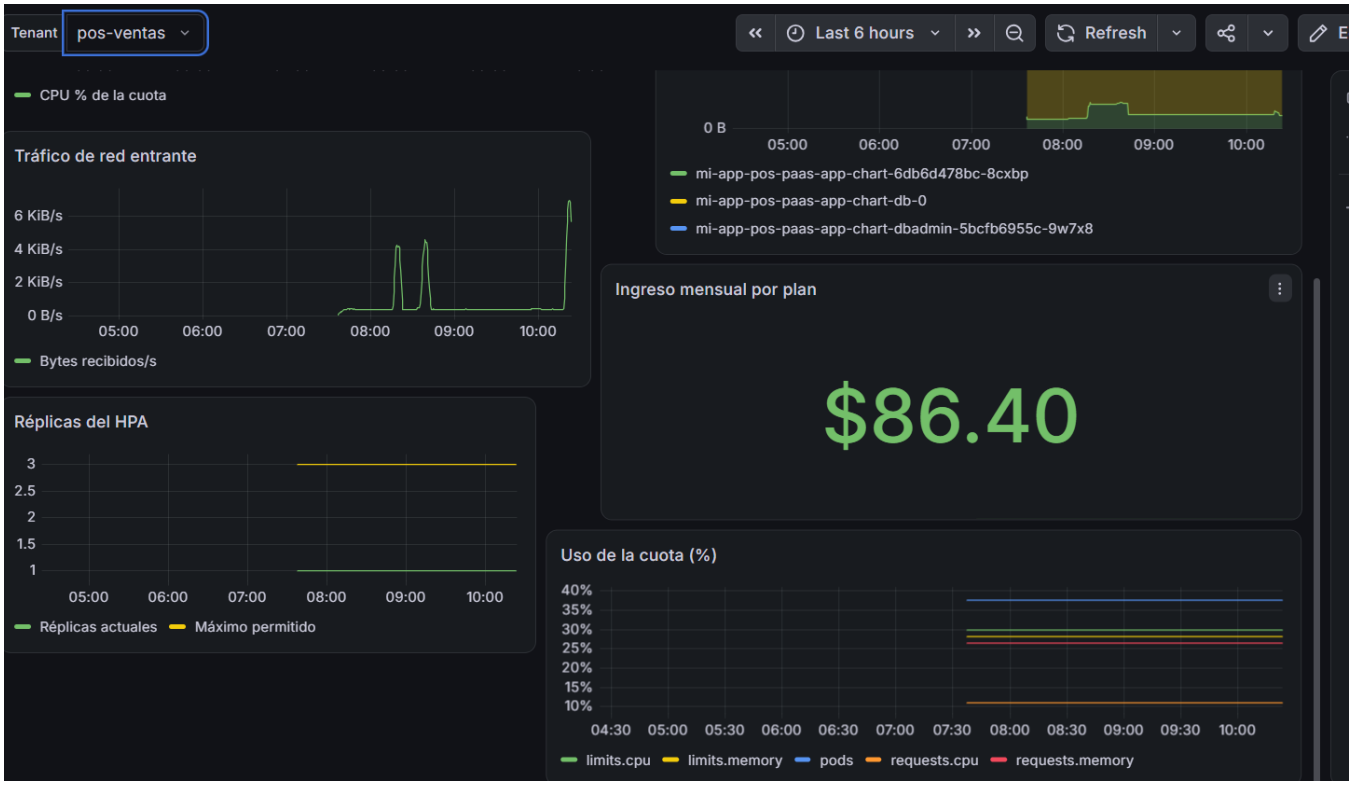


Figura 52. Al mismo tiempo, pos-ventas se mantiene en 1 réplica y su cuota no cambia: el escalado de un cliente no afecta al otro.

En el Cliente 2 la misma carga produjo algo distinto: su HPA se quedó en 1 réplica (su objetivo seguía en 70 %) y, al pasar de su límite del plan (**120 solicitudes/min por IP**), el Ingress empezó a responder **HTTP 503**. Es el middleware `limit-rpm` del plan Básico protegiendo al clúster, mientras que Alpha, con plan Pro (600 sol/min), absorbió la carga escalando.



Figura 53. Grafana · pos-ventas durante la misma prueba: 1 réplica fija y aparición de respuestas HTTP 503 (rojo) por el límite de solicitudes del plan Básico.

Momento	CPU / objetivo	Réplicas Alpha	Cuota Alpha (limits.cpu)	Réplicas Beta
Reposo (objetivo 70 %)	1–3 % / 70 %	2	≈ 40 %	1
Objetivo cambiado a 10 %	2 % / 10 %	2	≈ 40 %	1
Carga, minuto 1	15–18 % / 10 %	**4**	≈ 67 %	1
Carga, minuto 4	17 % / 10 %	**5** (máximo del plan)	≈ 80 %	1
Fin de la carga (≈ 10:25)	2–4 % / 10 %	5	≈ 80 %	1
5 min después (≈ 10:31)	3–4 % / 70 %	**2** (bajó solo)	≈ 40 %	1

Qué demuestra: la plataforma escala sola cuando sube el tráfico, **respeta el máximo del plan** (5 réplicas, dentro de la ResourceQuota de 10 pods) lo hace **sin afectar a otros clientes** y **vuelve a 2 réplicas** cuando baja el tráfico. Después de la prueba el objetivo se regresó a 70 % desde el panel.

14. Monitoreo y comparación de costos (Cliente 1 vs Cliente 2)

Se usaron dos dashboards de Grafana: **«PaaS por Tenant»** (del laboratorio, consumo contra cuota por tenant) y **«PaaS Parcial 2»** (consumo, solicitudes, latencia, facturación y comparativo). Para contar solicitudes por cliente se activaron las métricas del Ingress (`--enable-metrics=true`) y se agregó un ServiceMonitor (`monitoreo/ingress-metrics.yaml`).

14.1 Prueba de velocidad y de carga

Script `pruebas/carga.sh`: 20 peticiones HTTPS en reposo por cliente y luego 3 usuarios simultáneos por cliente, a los dos clientes a la vez. Resultados de dos corridas:

```
== Corrida 1 (120 s) ============================================================
Velocidad en reposo     TLS ms   Total ms   Max ms
alpha.paas.local          81.2      126.1    653.0
beta.paas.local           67.6      255.4    850.6
Carga                Peticiones  HTTP 200  HTTP 503  Acept/min   Prom ms
alpha.paas.local            482       482         0        241     441.0
beta.paas.local             266       266         0        133     977.6
== Corrida 2 (180 s, con métricas del Ingress) ====================================
alpha.paas.local     TLS 196.8 · Total 301.0 · Max 852.1     |  385 pet. · 385 OK · 128/min ·
713.4 ms
beta.paas.local      TLS 156.0 · Total 582.7 · Max 1091.5    |  290 pet. · 290 OK ·  97/min ·
1162.5 ms
```

Con la **misma carga**, el Cliente 1 atendió entre **1.3 y 1.8 veces más solicitudes** y respondió en **menos de la mitad** del tiempo en reposo. El Cliente 2 es más lento por su stack (PHP + Apache con MySQL) y porque su plan le da 1 réplica. En estas dos corridas no hubo respuestas 503: ningún cliente superó su límite de solicitudes con 3 usuarios (con 10 usuarios, en la sección 13, Beta sí llegó a su límite) (el límite admite ráfagas); el tope lo puso la velocidad de cada app.

14.2 Dashboards

Además de las capturas, se publicaron **snapshots** de los dashboards en Grafana (snapshots.raintank.io). Son copias de solo lectura con los datos reales de la prueba, que se pueden abrir sin acceso al clúster:

- **Snapshot 1:** https://snapshots.raintank.io/dashboard/snapshot/huJoq5WdpJWrukH1xKgUB5lf5eE1TLmC
- **Snapshot 2:** https://snapshots.raintank.io/dashboard/snapshot/tFk1QqEkFbtxYd98AOvbIqy7frWWTc4w
- **Snapshot 3:** https://snapshots.raintank.io/dashboard/snapshot/MtH8jq55CDBMEklOycikfEcOUv4c3j5W
- **Snapshot 4:** https://snapshots.raintank.io/dashboard/snapshot/GKpBBpqGnl194pjiMCxc1CuzDVsR4V5R
- **Snapshot 5:** https://snapshots.raintank.io/dashboard/snapshot/LqhVeLd8USPjSU3ecRHZo4YM9k7OHrli

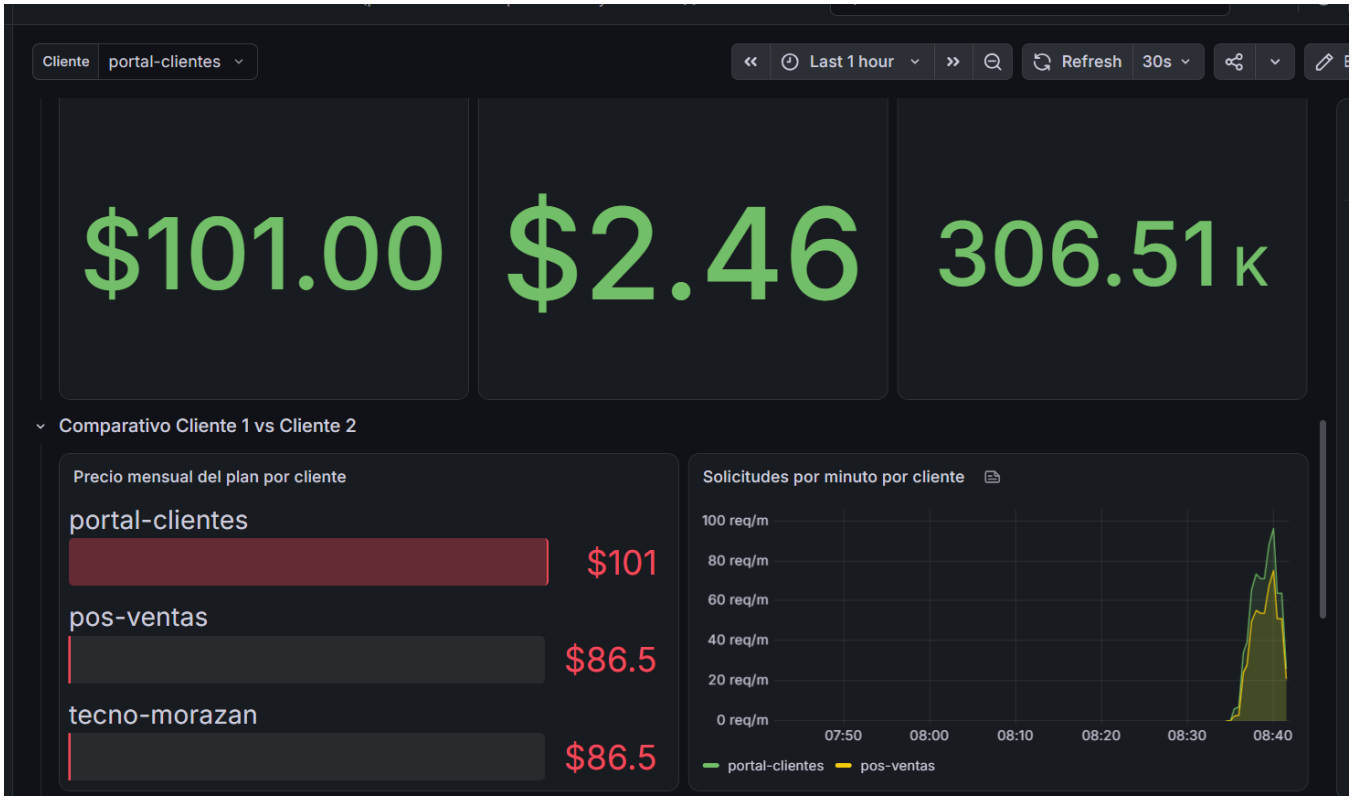


Figura 54. Cliente 1 (portal-clientes): precio del plan $101.00, costo por consumo $2.46, 306.5 mil solicitudes proyectadas y comparativo por cliente.

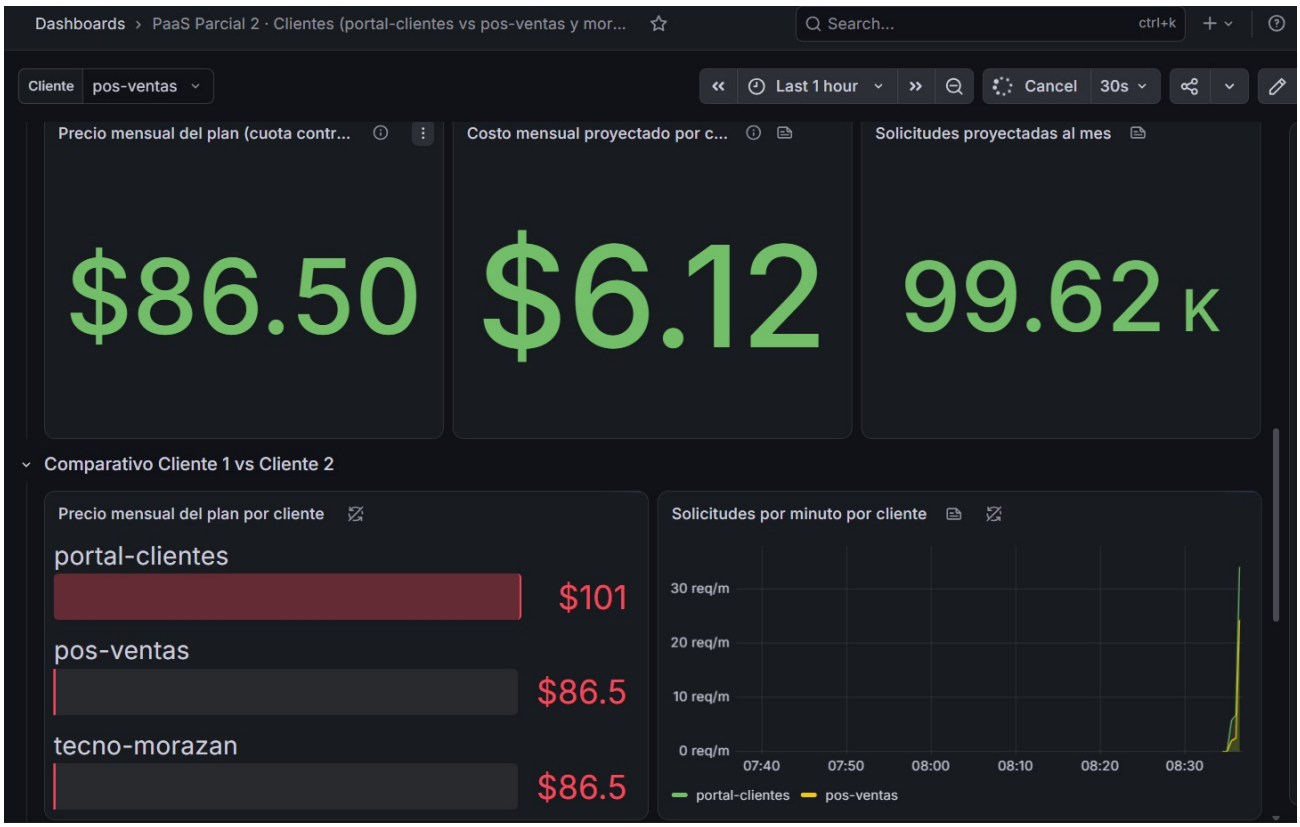


Figura 55. Cliente 2 (pos-ventas): precio del plan $86.50, costo por consumo $6.12 y 99.6 mil solicitudes proyectadas.

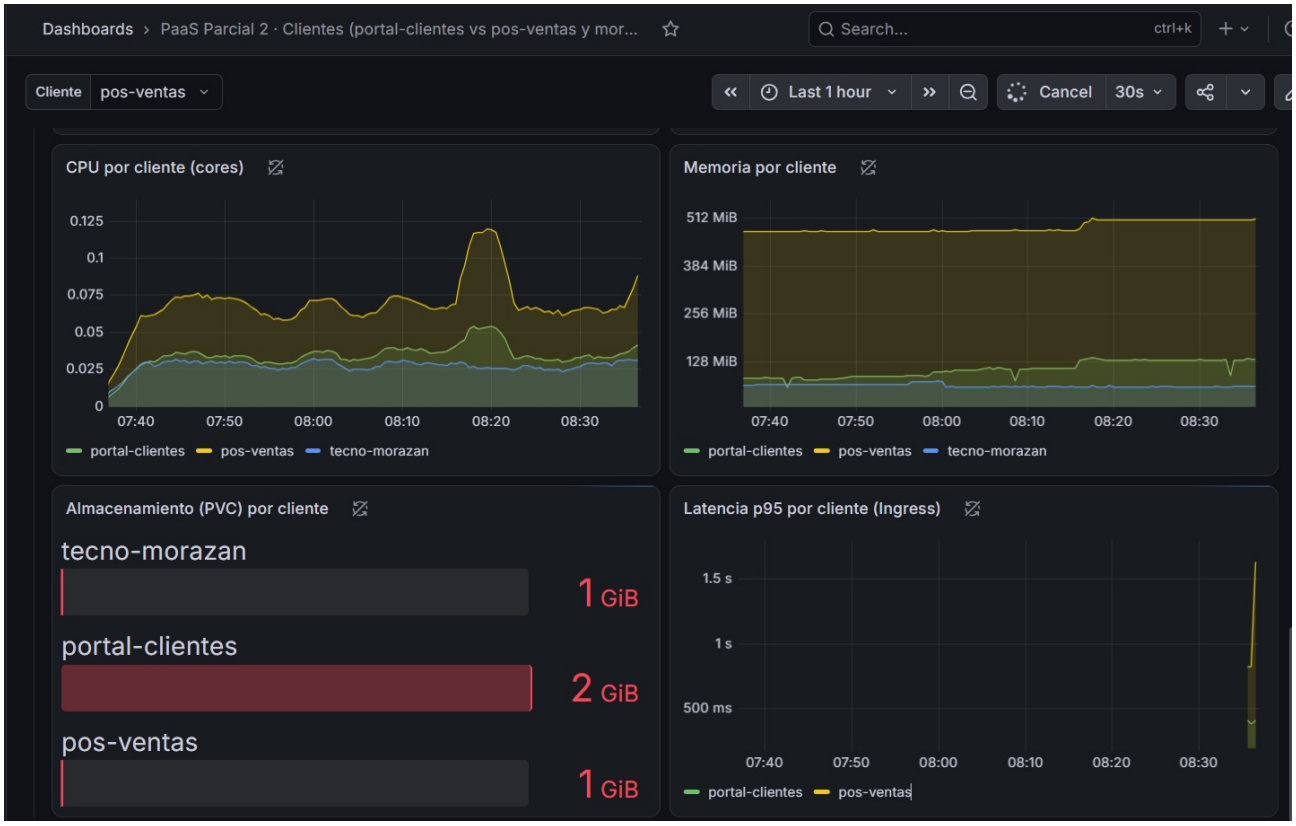


Figura 56. Comparativo durante la prueba: CPU, memoria, almacenamiento (PVC) y latencia p95 por cliente.

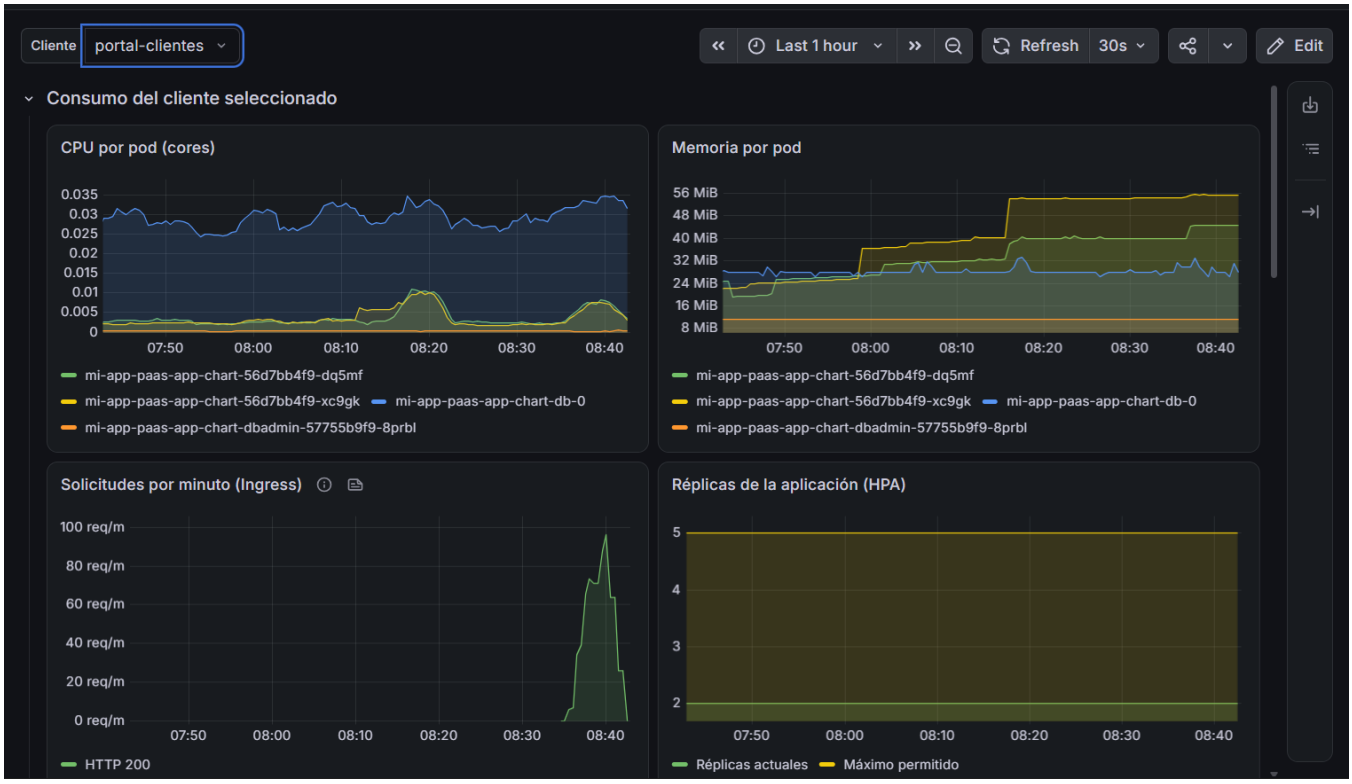


Figura 57. Cliente 1: consumo por pod, solicitudes por minuto, HPA, cuota, PVC 2 GiB y latencia p95.

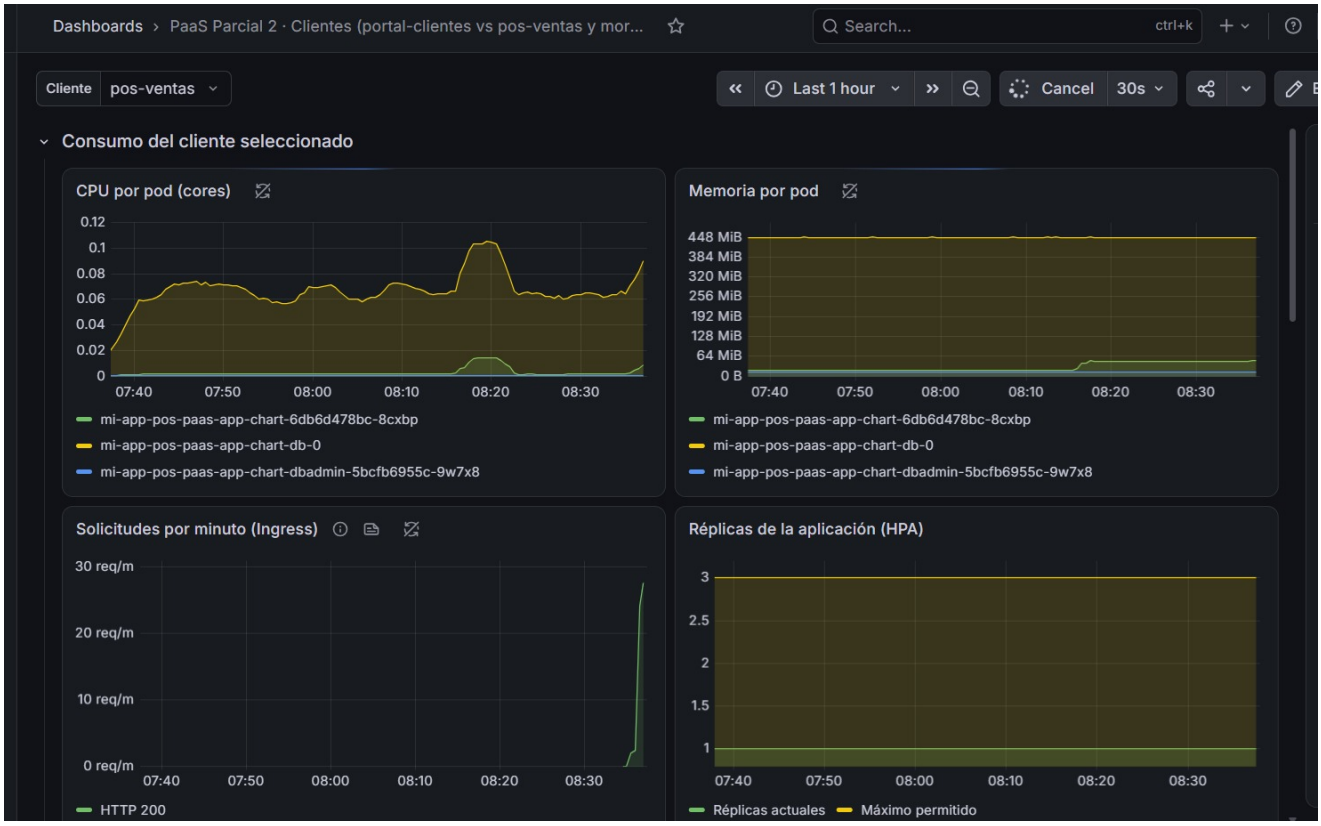


Figura 58. Cliente 2: la base MySQL domina el consumo de CPU y memoria.

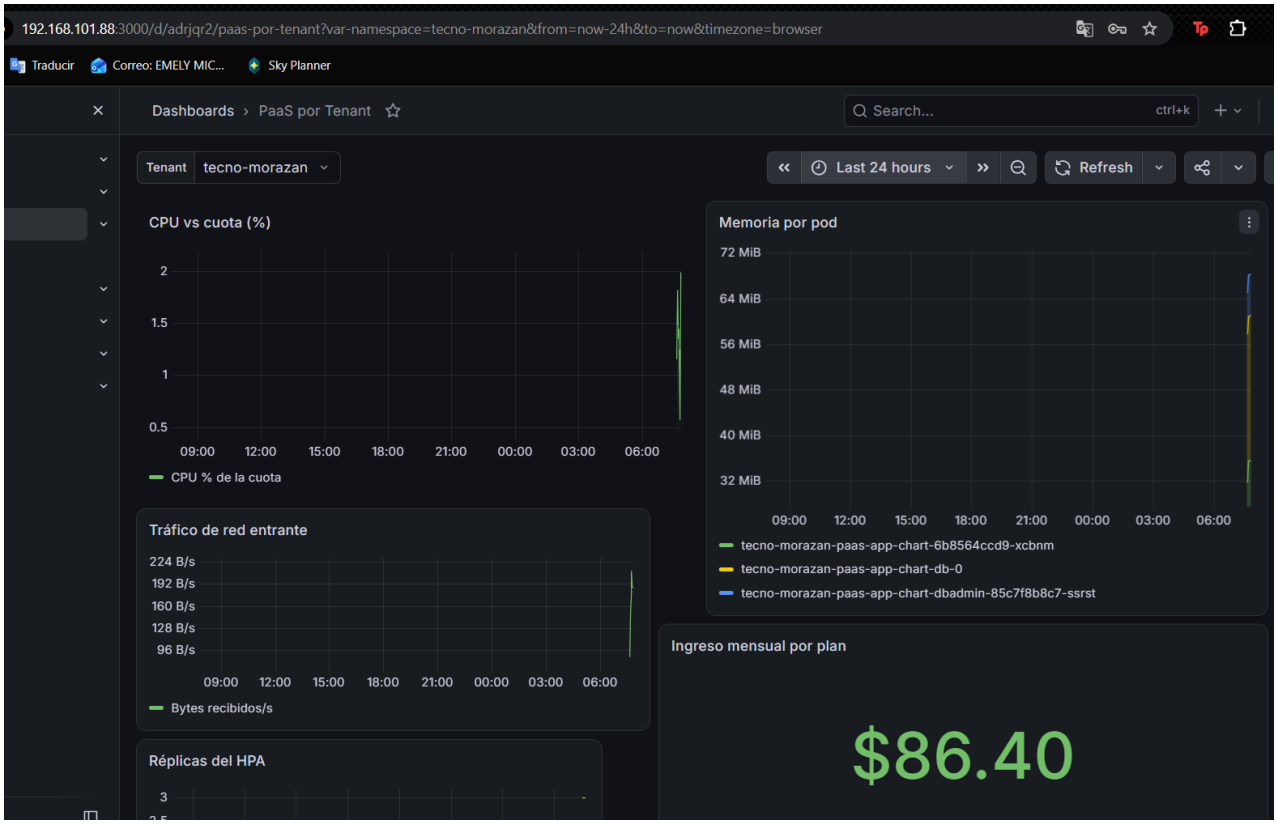


Figura 59. Dashboard del laboratorio «PaaS por Tenant» detectando también al cliente nuevo tecno-morazan (ingreso del plan $86.40).

14.3 Tabla comparativa de consumo y precio

Tarifas del Laboratorio 2: **$0.05 por core·hora** y **$0.01 por GiB·hora** (× 720 h/mes), más **$0.10 por GiB·mes** de disco y **$0.50 por millón de solicitudes**.

Métrica	Cliente 1 · portal-clientes (Pro)	Cliente 2 · pos-ventas (Básico)
Stack	Node.js + Express + PostgreSQL	PHP + Apache + MySQL
Cuota contratada (requests)	2 CPU · 4 GiB RAM · 10 pods	2 CPU · 2 GiB RAM · 8 pods
Réplicas de la app (HPA)	2 (máx. 5)	1 (máx. 3)
Solicitudes aceptadas por minuto (prueba)	**241** / 128	**133** / 97
Solicitudes por minuto (Grafana, pico, prom. 5 min)	~97	~70
Solicitudes proyectadas al mes	306.5 mil	99.6 mil
CPU usada (cores)	0.03 – 0.05	**0.06 – 0.12**
Memoria usada	~120 MiB	**~512 MiB** (MySQL ≈ 440 MiB)
Almacenamiento (PVC)	**2 GiB**	1 GiB
Velocidad en reposo (respuesta total)	**126 – 301 ms**	255 – 583 ms
Tiempo promedio bajo carga	**441 – 713 ms**	978 – 1163 ms
Latencia p95 bajo carga (Ingress)	1.17 – 2.20 s	1.63 – 2.44 s
Precio mensual del plan	(2×$0.05 + 4×$0.01)×720 + 2×$0.10 = **$101.00**	(2×$0.05 + 2×$0.01)×720 + 1×$0.10 = **$86.50**
Costo real proyectado por consumo	**$2.46** / mes	**$6.12** / mes
Costo de solicitudes	0.31 M × $0.50 = $0.15	0.10 M × $0.50 = $0.05
Margen de la plataforma	$101.00 − $2.46 = $98.54	$86.50 − $6.12 = $80.38

Conclusión. El Cliente 2 paga un plan más barato pero **consume más**: MySQL usa unas 4 veces más memoria y el doble de CPU que PostgreSQL, y además atiende menos solicitudes y responde más lento. Su costo real por consumo (**$6.12**) es 2.5 veces el del Cliente 1 (**$2.46**). El Cliente 1 paga más por su plan Pro (más RAM reservada, más réplicas y más disco), pero aprovecha mejor sus recursos. Para la plataforma, ambos planes cubren su consumo con margen; si el Cliente 2 creciera, convendría subirlo a Pro o ajustar la memoria de MySQL.

15. Incidencias y cómo se resolvieron

Problema	Causa	Solución
`helm upgrade` falló: webhook del Ingress «connection refused»	El controlador NGINX se reiniciaba (46 reinicios); el apiserver llegó a `Stopped`	Diagnóstico de la VM: RAM y disco bien, **carga promedio de 86 con 4 núcleos**; `minikube stop/start`
La carga seguía alta (66) tras reiniciar	El stack de monitoreo en un ciclo de reinicios (operator 265, node-exporter 146) saturaba etcd	Pausar el monitoreo durante los despliegues; Prometheus con retención de 12 h y recolección cada 60 s; Alertmanager apagado
La imagen solo existía en un nodo	`minikube image build --all` no copió a `minikube-m02`	Construir por nodo con `-n minikube` y `-n minikube-m02` (el panel lo hace solo)
MySQL tardó en estar listo	Descarga de 6 min e inicialización de InnoDB	readinessProbe de la app; el pod anterior siguió atendiendo
Paneles de solicitudes «No data»	El addon de minikube arranca NGINX sin `--enable-metrics`	Patch al Deployment con `--enable-metrics=true` + Service y ServiceMonitor
Navegador forzaba HTTPS	Cabecera HSTS de NGINX	`hsts: "false"` en el ConfigMap de

		ingress-nginx

16. Repositorio Git

Repositorio centralizado: **https://github.com/maxi300/paas-parcial2**. Es a la vez el entregable y la **fuente** desde la que NubePaaS construye y despliega.

```
paas-parcial2/
├── apps/
│   ├── alpha-node/      Dockerfile · package.json · src/server.js · src/api.js         (Cliente
1)
│   └── beta-php/        Dockerfile · src/*.php · src/lib/db.php · src/lib/layout.php
(Cliente 2)
├── chart/               Chart.yaml · values.yaml · values-portal.yaml · values-pos.yaml ·
templates/
├── panel/app.py         NubePaaS (portal del cliente + panel del dueño)
├── monitoreo/           ingress-metrics.yaml · dashboard-paas-parcial2.json
├── pruebas/             carga.sh · resultados-carga.txt
└── .gitignore           *.key *.pem *.csr *.srl certs/ node_modules/   ← ninguna llave ni
contraseña en Git
```

17. Conclusiones

- Un solo chart de Helm parametrizado basta para atender clientes muy distintos: cambiar el runtime, la base de datos, el plan o el middleware es cambiar el values.yaml.
- El modelo de autoservicio funciona: un cliente nuevo quedó en línea con HTTPS y base de datos propia en menos de 4 minutos, dando solo sus datos y su repositorio.
- El aislamiento se logra por capas: namespace, NetworkPolicy, ResourceQuota/LimitRange, Secrets propios y autorización en el portal.
- Las métricas permiten cobrar con datos reales: el plan más barato no siempre es el que menos consume; el stack (MySQL vs PostgreSQL) pesa en el costo.
- En un clúster pequeño, el monitoreo también consume: hubo que dimensionarlo (retención y frecuencia) para que no compitiera con los clientes.

Anexo A. Arranque de la plataforma

```
minikube start                                                  # 1) clúster
sudo kubectl --kubeconfig /home/max/.kube/config port-forward --address 0.0.0.0 \
     -n ingress-nginx svc/ingress-nginx-controller 80:80 443:443   # 2) acceso público
cd ~/paas-parcial2 && PANEL_PASSWORD='…' ALPHA_PASSWORD='…' BETA_PASSWORD='…' python3
panel/app.py   # 3) NubePaaS :5000
kubectl port-forward --address 0.0.0.0 -n monitoring svc/prometheus-grafana 3000:80
# 4) Grafana :3000
~/paas-parcial2/pruebas/carga.sh 180
# prueba de carga
```